

Deep Residual Learning for Image Recognition

Kaiming He Xiangyu Zhang Shaoqing Ren Jian Sun
Microsoft Research
{kahe, v-xiangz, v-shren, jiansun}@microsoft.com

Abstract

Deeper neural networks are more difficult to train. We present a residual learning framework to ease the training of networks that are substantially deeper than those used previously. We explicitly reformulate the layers as learning residual functions with reference to the layer inputs, instead of learning unreferenced functions. We provide comprehensive empirical evidence showing that these residual networks are easier to optimize, and can gain accuracy from considerably increased depth. On the ImageNet dataset we evaluate residual nets with a depth of up to 152 layers—8× deeper than VGG nets [41] but still having lower complexity. An ensemble of these residual nets achieves 3.57% error on the ImageNet test set. This result won the 1st place on the ILSVRC 2015 classification task. We also present analysis on CIFAR-10 with 100 and 1000 layers.

The depth of representations is of central importance for many visual recognition tasks. Solely due to our extremely deep representations, we obtain a 28% relative improvement on the COCO object detection dataset. Deep residual nets are foundations of our submissions to ILSVRC & COCO 2015 competitions¹, where we also won the 1st places on the tasks of ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation.

1. Introduction

Deep convolutional neural networks [22, 21] have led to a series of breakthroughs for image classification [21, 50, 40]. Deep networks naturally integrate low/mid/high-level features [50] and classifiers in an end-to-end multi-layer fashion, and the “levels” of features can be enriched by the number of stacked layers (depth). Recent evidence [41, 44] reveals that network depth is of crucial importance, and the leading results [41, 44, 13, 16] on the challenging ImageNet dataset [36] all exploit “very deep” [41] models, with a depth of sixteen [41] to thirty [16]. Many other non-trivial visual recognition tasks [8, 12, 7, 32, 27] have also

¹<http://image-net.org/challenges/LSVRC/2015/> and <http://mscoco.org/dataset/#detections-challenge2015>.

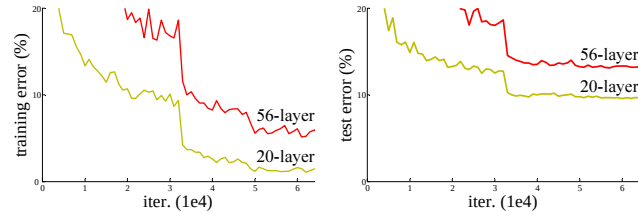


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer “plain” networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

greatly benefited from very deep models.

Driven by the significance of depth, a question arises: *Is learning better networks as easy as stacking more layers?* An obstacle to answering this question was the notorious problem of vanishing/exploding gradients [1, 9], which hamper convergence from the beginning. This problem, however, has been largely addressed by normalized initialization [23, 9, 37, 13] and intermediate normalization layers [16], which enable networks with tens of layers to start converging for stochastic gradient descent (SGD) with back-propagation [22].

When deeper networks are able to start converging, a degradation problem has been exposed: with the network depth increasing, accuracy gets saturated (which might be unsurprising) and then degrades rapidly. Unexpectedly, such degradation is *not caused by overfitting*, and adding more layers to a suitably deep model leads to *higher training error*, as reported in [11, 42] and thoroughly verified by our experiments. Fig. 1 shows a typical example.

The degradation (of training accuracy) indicates that not all systems are similarly easy to optimize. Let us consider a shallower architecture and its deeper counterpart that adds more layers onto it. There exists a solution *by construction* to the deeper model: the added layers are *identity* mapping, and the other layers are copied from the learned shallower model. The existence of this constructed solution indicates that a deeper model should produce no higher training error than its shallower counterpart. But experiments show that our current solvers on hand are unable to find solutions that

用于图像识别的深度残差学习

何恺明 张祥雨 任少卿 孙剑微软亚洲研究院{kahe, v-xiangz, v-shren, jiansun}@microsoft.com

摘要

更深的神经网络更难以训练。我们提出了一种残差学习框架，以便更轻松地训练比以往深得多的网络。我们明确地将各层重新表述为在层输入的基础上学习残差函数，而非学习无参照的函数。我们提供了全面的实验依据，表明这些残差网络更易于优化，并能从大幅增加的深度中获得更高的准确率。在 *ImageNet* 数据集上，我们评估了深度多达 152 层的残差网络——8×比 *VGG* 网络更深 [41] 但复杂度反而更低。这些残差网络的集成在 *ImageNet* test集上取得了 3.57% 的错误率。该结果在 *ILSVRC 2015* 分类任务中获得了第一名。我们还在 *CIFAR-10* 上对 100 层和 1000 层的网络进行了分析。

表征的深度对于许多视觉识别任务至关重要。仅凭我们极其深层的表征，我们在 *COCO* 目标检测数据集上就获得了 28% 的相对提升。深度残差网络是我们参加 *ILSVRC & COCO 2015* 竞赛¹，提交作品的基础，我们在该竞赛中同时获得了 *ImageNet* 检测、*ImageNet* 定位、*COCO* 检测和 *COCO* 分割四项任务的第 1 名。

1. 引言

深度卷积神经网络 [22, 21] 在图像分类 [21, 50, 40] 方面带来了一系列突破。深度网络以端到端的多层方式自然地整合了低层/中层/高层 [50] 特征和分类器，特征的“层次”可以通过堆叠层数（深度）来丰富。最近的研究[41, 44] 表明，网络深度至关重要，挑战性的 ImageNet 数据集 [36] 上的领先成果 [41, 44, 13, 16] 都利用了“非常深”的 [41] 模型，深度达到十六层 [41] 至三十层 [16]。许多其他非 [8, 12, 7, 32, 27] 平凡的视觉识别任务也

¹<http://image-net.org/challenges/LSVRC/2015/> 和 <http://mscoco.org/dataset/#detections-challenge2015>。

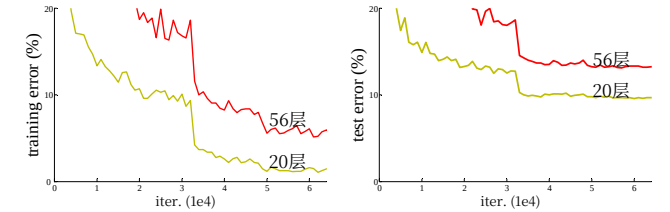


图1. 在 CIFAR-10 数据集上使用 20 层和 56 层“普通”网络的训练误差（左）和测试误差（右）。更深的网络具有更高的训练误差，因此测试误差也更高。ImageNet 上的类似现象见图 4。

从非常深的模型中获益匪浅。

受深度重要性的启发，随之产生了一个问题：学习更好的网络是否就像堆叠更多的层那样简单？回答这一问题的一个障碍是臭名昭著的梯度消失/梯度爆炸问题 [1, 9]，它从一开始就阻碍了收敛。然而，归一化初始化 [23, 9, 37, 13] 以及中间归一化层 [16]，在很大程度上解决了这个问题，它们使具有数十层的网络能够通过基于反向传播的随机梯度下降（SGD）开始收敛 [22]。

当更深的网络能够开始收敛时，一个退化问题便暴露出来：随着网络深度的增加，准确率趋于饱和（这并不令人意外），随后便迅速下降。出人意料的是，这种退化并非由过拟合引起，并且在适当深度的模型中增加更多层反而会导致更高的训练误差，正如 [11, 42] 所报告的，并已被我们的实验充分验证。图 1 展示了一个典型示例。

这种（训练准确率的）退化现象表明，并非所有系统都同样易于优化。让我们考虑一个较浅的架构，以及在其基础上增加更多层的更深的对应模型。对于更深的模型，按构造方式存在一个解：所添加的层为恒等映射，其余各层则从已学习的较浅模型中复制而来。这个构造解的存在表明，更深模型的训练误差不应高于其较浅的对应模型。但实验表明，我们目前手头的求解器无法找到能够

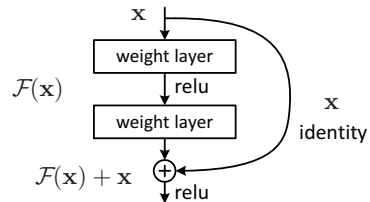


Figure 2. Residual learning: a building block.

are comparably good or better than the constructed solution (or unable to do so in feasible time).

In this paper, we address the degradation problem by introducing a *deep residual learning* framework. Instead of hoping each few stacked layers directly fit a desired underlying mapping, we explicitly let these layers fit a residual mapping. Formally, denoting the desired underlying mapping as $\mathcal{H}(\mathbf{x})$, we let the stacked nonlinear layers fit another mapping of $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$. The original mapping is recast into $\mathcal{F}(\mathbf{x}) + \mathbf{x}$. We hypothesize that it is easier to optimize the residual mapping than to optimize the original, unreferenced mapping. To the extreme, if an identity mapping were optimal, it would be easier to push the residual to zero than to fit an identity mapping by a stack of nonlinear layers.

The formulation of $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ can be realized by feedforward neural networks with “shortcut connections” (Fig. 2). Shortcut connections [2, 34, 49] are those skipping one or more layers. In our case, the shortcut connections simply perform *identity* mapping, and their outputs are added to the outputs of the stacked layers (Fig. 2). Identity shortcut connections add neither extra parameter nor computational complexity. The entire network can still be trained end-to-end by SGD with backpropagation, and can be easily implemented using common libraries (*e.g.*, Caffe [19]) without modifying the solvers.

We present comprehensive experiments on ImageNet [36] to show the degradation problem and evaluate our method. We show that: 1) Our extremely deep residual nets are easy to optimize, but the counterpart “plain” nets (that simply stack layers) exhibit higher training error when the depth increases; 2) Our deep residual nets can easily enjoy accuracy gains from greatly increased depth, producing results substantially better than previous networks.

Similar phenomena are also shown on the CIFAR-10 set [20], suggesting that the optimization difficulties and the effects of our method are not just akin to a particular dataset. We present successfully trained models on this dataset with over 100 layers, and explore models with over 1000 layers.

On the ImageNet classification dataset [36], we obtain excellent results by extremely deep residual nets. Our 152-layer residual net is the deepest network ever presented on ImageNet, while still having lower complexity than VGG nets [41]. Our ensemble has **3.57%** top-5 error on the

ImageNet *test* set, and *won the 1st place in the ILSVRC 2015 classification competition*. The extremely deep representations also have excellent generalization performance on other recognition tasks, and lead us to further *win the 1st places on: ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation* in ILSVRC & COCO 2015 competitions. This strong evidence shows that the residual learning principle is generic, and we expect that it is applicable in other vision and non-vision problems.

2. Related Work

Residual Representations. In image recognition, VLAD [18] is a representation that encodes by the residual vectors with respect to a dictionary, and Fisher Vector [30] can be formulated as a probabilistic version [18] of VLAD. Both of them are powerful shallow representations for image retrieval and classification [4, 48]. For vector quantization, encoding residual vectors [17] is shown to be more effective than encoding original vectors.

In low-level vision and computer graphics, for solving Partial Differential Equations (PDEs), the widely used Multigrid method [3] reformulates the system as subproblems at multiple scales, where each subproblem is responsible for the residual solution between a coarser and a finer scale. An alternative to Multigrid is hierarchical basis preconditioning [45, 46], which relies on variables that represent residual vectors between two scales. It has been shown [3, 45, 46] that these solvers converge much faster than standard solvers that are unaware of the residual nature of the solutions. These methods suggest that a good reformulation or preconditioning can simplify the optimization.

Shortcut Connections. Practices and theories that lead to shortcut connections [2, 34, 49] have been studied for a long time. An early practice of training multi-layer perceptrons (MLPs) is to add a linear layer connected from the network input to the output [34, 49]. In [44, 24], a few intermediate layers are directly connected to auxiliary classifiers for addressing vanishing/exploding gradients. The papers of [39, 38, 31, 47] propose methods for centering layer responses, gradients, and propagated errors, implemented by shortcut connections. In [44], an “inception” layer is composed of a shortcut branch and a few deeper branches.

Concurrent with our work, “highway networks” [42, 43] present shortcut connections with gating functions [15]. These gates are data-dependent and have parameters, in contrast to our identity shortcuts that are parameter-free. When a gated shortcut is “closed” (approaching zero), the layers in highway networks represent *non-residual* functions. On the contrary, our formulation always learns residual functions; our identity shortcuts are never closed, and all information is always passed through, with additional residual functions to be learned. In addition, high-

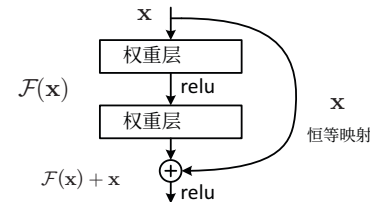


图2. 残差学习：一个构建块。

与该构造解相当或更优的解（或无法在可行时间内找到这样的解）。

在本文中，我们通过引入深度残差学习框架来解决退化问题。我们不再期望每几个堆叠层直接拟合一个期望的基础映射，而是显式地让这些层拟合一个残差映射。形式上，将期望的基础映射表示为 $\mathcal{H}(\mathbf{x})$ ，我们让堆叠的非线性层拟合另一个关于 $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$ 的映射。原始映射被转化为 $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ 。我们假设优化残差映射比优化原始的、未参考的映射更容易。极端情况下，如果恒等映射是最优的，那么将残差推至零比用一堆非线性层来拟合恒等映射要容易得多。

$\mathcal{F}(\mathbf{x}) + \mathbf{x}$ 的公式可以通过带有“快捷连接”的前馈神经网络来实现（图2）。快捷连接 [2, 34, 49] 是指跳过一层或多层的连接。在我们的情况下，快捷连接简单地执行恒等映射，其输出被加到堆叠层的输出上

（图2）。恒等快捷连接既不增加额外参数，也不增加计算复杂度。整个网络仍然可以通过SGD配合反向传播进行端到端训练，并且可以使用常见库（例如，Caffe [19]）轻松实现，无需修改求解器。

我们在ImageNet[36] 上进行了全面实验，以展示退化问题并评估我们的方法。我们表明：1) 我们的超深残差网络易于优化，而对应的“普通”网络（仅简单堆叠层）在深度增加时会表现出更高的训练误差；2) 我们的深度残差网络能够轻松地大幅增加的深度中获得精度提升，所产生的结果显著优于之前的网络。

在CIFAR-10数据集[20], 上也呈现类似现象，表明我们的优化难题和方法效果并非仅与某一特定数据集相关。我们成功地在该数据集上训练了超过100层的模型，并探索了超过1000层的模型。

在ImageNet分类数据集 [36], 上，我们通过极深的残差网络取得了优异的结果。我们的152层残差网络是ImageNet上迄今提出的最深网络，同时复杂度仍低于VGG网络 [41]。我们的集成模型在ImageNet测试集上的top-5错误率为**3.57%**，

ImageNet 测试集上取得了*ILSVRC 2015*分类竞赛第一名的成绩。极深的表示在其他识别任务上也具有出色的泛化性能，使我们进一步在*ILSVRC与COCO 2015*竞赛中斩获：*ImageNet*检测、*ImageNet*定位、*COCO*检测和*COCO*分割的多项第一名。这些有力的证据表明残差学习原理具有普适性，我们期望它能够应用于其他视觉和非视觉问题。

2. 相关工作

残差表示。在图像识别中，VLAD[18]是一种通过相对于字典的残差向量进行编码的表示方法，而Fisher向量 [30] 可以被表述为VLAD的概率版本 [18]。两者都是用于图像检索和分类的强大浅层表示 [4, 48]。在向量量化中，编码残差向量 [17] 被证明比编码原始向量更为有效。

在底层视觉和计算机图形学中，为了求解偏微分方程（PDEs），广泛使用的多重网格方法 [3] 将系统重新表述为多个尺度上的子问题，其中每个子问题负责较粗尺度与较细尺度之间的残差解。多重网格方法的一种替代方案是层次基预条件法 [45, 46]，该方法依赖于表示两个尺度之间残差向量的变量。已有研究 [3, 45, 46]表明，这些求解器的收敛速度远快于不了解解的残差性质的标准求解器。这些方法表明，良好的重新表述或预条件处理可以简化优化过程。

捷径连接。引导捷径连接 [2, 34, 49]的实践与理论已被研究了很长时间。训练多层感知机（MLP）的一种早期做法是将一个线性层从网络输入直接连接到输出 [34, 49]。在 [44, 24], 中，若干中间层直接连接到辅助分类器，以解决梯度消失/梯度爆炸问题。[39, 38, 31, 47]的论文提出了对层响应、梯度和传播误差进行中心化的方法，这些方法通过捷径连接实现。在 [44], 中，一个“inception”层由一个捷径分支和若干更深的分支组成。

与我们的工作同时期，“highway networks” [42, 43]提出了带有门控函数的快捷连接 [15]。这些门控是数据依赖的且包含参数，与我们无参数的恒等捷径不同。当带门控的捷径“关闭”（趋近于零）时，highway网络中的层表示非残差函数。相反，我们的公式始终学习残差函数；我们的恒等捷径从不关闭，所有信息始终通过，同时还需学习额外的残差函数。此外，high-

way networks have not demonstrated accuracy gains with extremely increased depth (*e.g.*, over 100 layers).

3. Deep Residual Learning

3.1. Residual Learning

Let us consider $\mathcal{H}(\mathbf{x})$ as an underlying mapping to be fit by a few stacked layers (not necessarily the entire net), with $\mathbf{x}$ denoting the inputs to the first of these layers. If one hypothesizes that multiple nonlinear layers can asymptotically approximate complicated functions², then it is equivalent to hypothesize that they can asymptotically approximate the residual functions, *i.e.*, $\mathcal{H}(\mathbf{x}) - \mathbf{x}$ (assuming that the input and output are of the same dimensions). So rather than expect stacked layers to approximate $\mathcal{H}(\mathbf{x})$, we explicitly let these layers approximate a residual function $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$. The original function thus becomes $\mathcal{F}(\mathbf{x}) + \mathbf{x}$. Although both forms should be able to asymptotically approximate the desired functions (as hypothesized), the ease of learning might be different.

This reformulation is motivated by the counterintuitive phenomena about the degradation problem (Fig. 1, left). As we discussed in the introduction, if the added layers can be constructed as identity mappings, a deeper model should have training error no greater than its shallower counterpart. The degradation problem suggests that the solvers might have difficulties in approximating identity mappings by multiple nonlinear layers. With the residual learning reformulation, if identity mappings are optimal, the solvers may simply drive the weights of the multiple nonlinear layers toward zero to approach identity mappings.

In real cases, it is unlikely that identity mappings are optimal, but our reformulation may help to precondition the problem. If the optimal function is closer to an identity mapping than to a zero mapping, it should be easier for the solver to find the perturbations with reference to an identity mapping, than to learn the function as a new one. We show by experiments (Fig. 7) that the learned residual functions in general have small responses, suggesting that identity mappings provide reasonable preconditioning.

3.2. Identity Mapping by Shortcuts

We adopt residual learning to every few stacked layers. A building block is shown in Fig. 2. Formally, in this paper we consider a building block defined as:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}. \quad (1)$$

Here $\mathbf{x}$ and $\mathbf{y}$ are the input and output vectors of the layers considered. The function $\mathcal{F}(\mathbf{x}, \{W_i\})$ represents the residual mapping to be learned. For the example in Fig. 2 that has two layers, $\mathcal{F} = W_2\sigma(W_1\mathbf{x})$ in which σ denotes

ReLU [29] and the biases are omitted for simplifying notations. The operation $\mathcal{F} + \mathbf{x}$ is performed by a shortcut connection and element-wise addition. We adopt the second nonlinearity after the addition (*i.e.*, $\sigma(\mathbf{y})$, see Fig. 2).

The shortcut connections in Eqn.(1) introduce neither extra parameter nor computation complexity. This is not only attractive in practice but also important in our comparisons between plain and residual networks. We can fairly compare plain/residual networks that simultaneously have the same number of parameters, depth, width, and computational cost (except for the negligible element-wise addition).

The dimensions of $\mathbf{x}$ and $\mathcal{F}$ must be equal in Eqn.(1). If this is not the case (*e.g.*, when changing the input/output channels), we can perform a linear projection W_s by the shortcut connections to match the dimensions:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s\mathbf{x}. \quad (2)$$

We can also use a square matrix W_s in Eqn.(1). But we will show by experiments that the identity mapping is sufficient for addressing the degradation problem and is economical, and thus W_s is only used when matching dimensions.

The form of the residual function $\mathcal{F}$ is flexible. Experiments in this paper involve a function $\mathcal{F}$ that has two or three layers (Fig. 5), while more layers are possible. But if $\mathcal{F}$ has only a single layer, Eqn.(1) is similar to a linear layer: $\mathbf{y} = W_1\mathbf{x} + \mathbf{x}$, for which we have not observed advantages.

We also note that although the above notations are about fully-connected layers for simplicity, they are applicable to convolutional layers. The function $\mathcal{F}(\mathbf{x}, \{W_i\})$ can represent multiple convolutional layers. The element-wise addition is performed on two feature maps, channel by channel.

3.3. Network Architectures

We have tested various plain/residual nets, and have observed consistent phenomena. To provide instances for discussion, we describe two models for ImageNet as follows.

Plain Network. Our plain baselines (Fig. 3, middle) are mainly inspired by the philosophy of VGG nets [41] (Fig. 3, left). The convolutional layers mostly have 3×3 filters and follow two simple design rules: (i) for the same output feature map size, the layers have the same number of filters; and (ii) if the feature map size is halved, the number of filters is doubled so as to preserve the time complexity per layer. We perform downsampling directly by convolutional layers that have a stride of 2. The network ends with a global average pooling layer and a 1000-way fully-connected layer with softmax. The total number of weighted layers is 34 in Fig. 3 (middle).

It is worth noticing that our model has *fewer* filters and *lower* complexity than VGG nets [41] (Fig. 3, left). Our 34-layer baseline has 3.6 billion FLOPs (multiply-adds), which is only 18% of VGG-19 (19.6 billion FLOPs).

way网络尚未证明在极深网络（例如，超过100层）中具有精度提升。

3. 深度残差学习

3.1. 残差学习

让我们将 $\mathcal{H}(\mathbf{x})$ 视为一个待由少量堆叠层（不一定是整个网络）拟合的基础映射，其中 $\mathbf{x}$ 表示这些层中第一层的输入。如果假设多个非线性层能够渐近地逼近复杂函数²，那么这就等价于假设它们能够渐近地逼近残差函数，即， $\mathcal{H}(\mathbf{x}) - \mathbf{x}$ （假设输入和输出具有相同的维度）。因此，我们不期望堆叠层去逼近 $\mathcal{H}(\mathbf{x})$ ，而是明确地让这些层去逼近残差函数 $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$ 。这样，原始函数就变为 $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ 。尽管两种形式都应该能够渐近地逼近所需的函数（如假设的那样），但它们的难易程度可能不同。

这种重构的动机来源于关于退化问题的反直觉现象（图1，左侧）。正如我们在引言中所讨论的，如果所添加的层可以被构造成恒等映射，那么更深的模型其训练误差不应大于较浅的模型。退化问题表明，求解器可能在用多个非线性层逼近恒等映射时遇到困难。采用残差学习重构方式后，如果恒等映射是最优的，求解器只需将多个非线性层的权重推向零，即可逼近恒等映射。

在实际情况下，恒等映射不太可能是最优的，但我们的重构方式可能有助于为问题提供预处理条件。如果最优函数更接近恒等映射而非零映射，那么求解器以恒等映射为参考来寻找扰动，应该比将其作为一个全新函数来学习更加容易。我们通过实验（图7）表明，学习到的残差函数通常具有较小的响应值，这说明恒等映射能够提供合理的预处理条件。

3.2. 通过快捷连接实现恒等映射

我们在每隔几层堆叠的层上采用残差学习。图2展示了一个构建块。形式化地，在本文中我们考虑如下定义的构建块：

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}. \quad (1)$$

其中 $\mathbf{x}$ 和 $\mathbf{y}$ 分别为所考虑层的输入和输出向量。函数 $\mathcal{F}(\mathbf{x}, \{W_i\})$ 表示需要学习的残差映射。以图 2 中具有两层的示例为例， $\mathcal{F} = W_2\sigma(W_1\mathbf{x})$ ，其中 σ 表示

²然而，这一假设仍然是一个开放性问题。参见 [28]。

ReLU [29]，为简化符号而省略了偏置项。操作 $\mathcal{F} + \mathbf{x}$ 通过捷径连接和逐元素加法实现。我们在加法之后采用第二个非线性函数（即， $\sigma(\mathbf{y})$ ，见图 2）。

公式(1)中的快捷连接既不引入额外参数，也不增加计算复杂度。这不仅在实践中很有吸引力，而且在我们对比普通网络和残差网络时也非常重要。我们可以公平地对比在参数数量、深度、宽度和计算成本（除可忽略的逐元素加法外）均相同的普通网络和残差网络。

在公式(1)中， $\mathbf{x}$ 和 $\mathcal{F}$ 的维度必须相同。如果不是这种情况（例如，当改变输入/输出通道数时），我们可以通过捷径连接进行线性变换 W_s ，以使维度匹配：

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s\mathbf{x}. \quad (2)$$

我们也可以在公式(1)中使用方阵 W_s 。但我们将通过实验证明，恒等映射足以解决退化问题，且更为经济，因此 W_s 仅在维度需要匹配时使用。

残差函数 $\mathcal{F}$ 的形式是灵活的。本文的实验涉及具有两层或三层的函数 $\mathcal{F}$ （图5），同时也可以有更多的层。但如果 $\mathcal{F}$ 只有一层，则公式(1)类似于线性层： $\mathbf{y} = W_1\mathbf{x} + \mathbf{x}$ ，我们尚未观察到其优势。

我们还注意到，尽管上述符号是针对全连接层的简化表示，但它们同样适用于卷积层。函数 $\mathcal{F}(\mathbf{x}, \{W_i\})$ 可以表示多个卷积层。逐元素相加是在两张特征图上按通道逐个进行的。

3.3. 网络架构

我们测试了多种普通网络和残差网络，并观察到了一致的现象。为了提供讨论的实例，我们对ImageNet描述了两个模型，如下所述。

普通网络。 我们的普通基线网络（图3，中间）主要受到VGG网络 [41]（图3，左边）设计思想的启发。卷积层大多具有 3×3 个滤波器，并遵循两条简单的设计准则：(i) 对于相同的输出特征图尺寸，各层使用相同数量的滤波器；(ii) 如果特征图尺寸减半，则滤波器数量加倍，以保持每层的时间复杂度。我们直接通过步长为2的卷积层进行下采样。网络以一个全局平均池化层和一个具有softmax的1000类全连接层结束。在图3（中间）中，加权层的总数为34层。

值得注意的是，我们的模型比VGG网络 [41]（图3，左）具有更少的滤波器和更低的复杂度。我们的34层基线网络的FLOPs（乘加运算）为36亿次，仅为VGG-19（196亿次FLOPs）的18%。

²This hypothesis, however, is still an open question. See [28].

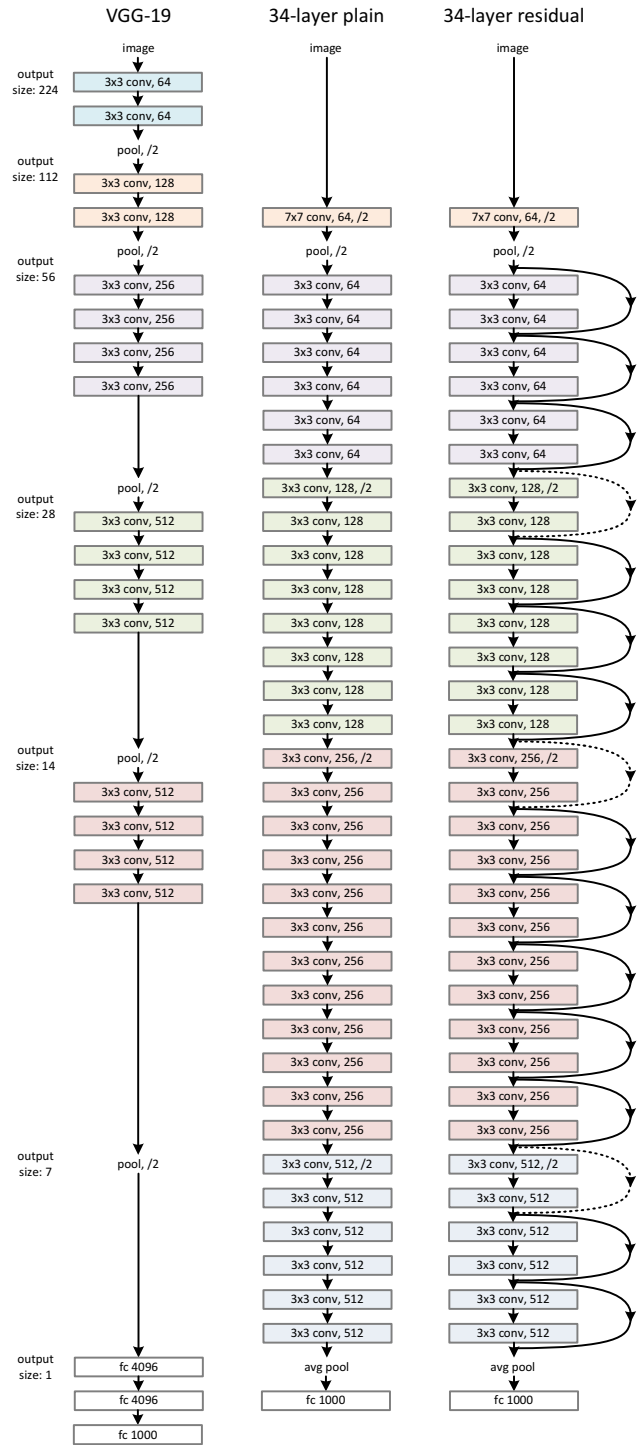


Figure 3. Example network architectures for ImageNet. **Left:** the VGG-19 model [41] (19.6 billion FLOPs) as a reference. **Middle:** a plain network with 34 parameter layers (3.6 billion FLOPs). **Right:** a residual network with 34 parameter layers (3.6 billion FLOPs). The dotted shortcuts increase dimensions. **Table 1** shows more details and other variants.

Residual Network. Based on the above plain network, we insert shortcut connections (Fig. 3, right) which turn the network into its counterpart residual version. The identity shortcuts (Eqn.(1)) can be directly used when the input and output are of the same dimensions (solid line shortcuts in Fig. 3). When the dimensions increase (dotted line shortcuts in Fig. 3), we consider two options: (A) The shortcut still performs identity mapping, with extra zero entries padded for increasing dimensions. This option introduces no extra parameter; (B) The projection shortcut in Eqn.(2) is used to match dimensions (done by 1×1 convolutions). For both options, when the shortcuts go across feature maps of two sizes, they are performed with a stride of 2.

3.4. Implementation

Our implementation for ImageNet follows the practice in [21, 41]. The image is resized with its shorter side randomly sampled in [256, 480] for scale augmentation [41]. A 224×224 crop is randomly sampled from an image or its horizontal flip, with the per-pixel mean subtracted [21]. The standard color augmentation in [21] is used. We adopt batch normalization (BN) [16] right after each convolution and before activation, following [16]. We initialize the weights as in [13] and train all plain/residual nets from scratch. We use SGD with a mini-batch size of 256. The learning rate starts from 0.1 and is divided by 10 when the error plateaus, and the models are trained for up to 60×10^4 iterations. We use a weight decay of 0.0001 and a momentum of 0.9. We do not use dropout [14], following the practice in [16].

In testing, for comparison studies we adopt the standard 10-crop testing [21]. For best results, we adopt the fully-convolutional form as in [41, 13], and average the scores at multiple scales (images are resized such that the shorter side is in $\{224, 256, 384, 480, 640\}$).

4. Experiments

4.1. ImageNet Classification

We evaluate our method on the ImageNet 2012 classification dataset [36] that consists of 1000 classes. The models are trained on the 1.28 million training images, and evaluated on the 50k validation images. We also obtain a final result on the 100k test images, reported by the test server. We evaluate both top-1 and top-5 error rates.

Plain Networks. We first evaluate 18-layer and 34-layer plain nets. The 34-layer plain net is in Fig. 3 (middle). The 18-layer plain net is of a similar form. See Table 1 for detailed architectures.

The results in Table 2 show that the deeper 34-layer plain net has higher validation error than the shallower 18-layer plain net. To reveal the reasons, in Fig. 4 (left) we compare their training/validation errors during the training procedure. We have observed the degradation problem - the



图3. ImageNet的网络架构示例。左：作为参照的VGG-19模型 [41]（196亿次浮点运算）。中：一个具有34个参数层的普通网络（36亿次浮点运算）。右：一个具有34个参数层的残差网络（36亿次浮点运算）。虚线快捷连接用于增加维度。表1展示了更多细节和其他变体。

残差网络。在上述普通网络的基础上，我们插入了快捷连接（图3，右），使其转变为对应的残差版本。当输入和输出维度相同时，可直接使用恒等快捷连接（公式(1)，图3中的实线快捷连接）。当维度增加时（图3中的虚线快捷连接），我们考虑两种方案：(A) 快捷连接仍执行恒等映射，额外维度以零填充。此方案不引入任何额外参数；(B) 使用公式(2)中的投影快捷连接来匹配维度（由 1×1 卷积实现）。对于这两种方案，当快捷连接跨越不同大小的特征图时，均以步长2执行。

3.4. 实现细节

我们在ImageNet上的实现遵循 [21, 41]中的做法。图像被调整大小，其较短边在 [256, 480] 范围内随机采样，以进行尺度增强 [41]。从图像或其水平翻转图像中随机采样一个 224×224 裁剪区域，并减去逐像素均值 [21]。使用 [21]中的标准颜色增强方法。我们按照 [16]的做法，在每次卷积之后、激活之前采用批量归一化（BN） [16]。我们按照 [13] 的方法初始化权重，并从头开始训练所有普通网络和残差网络。我们使用小批量大小为256的SGD。学习率从0.1开始，当误差趋于平稳时除以10，模型最多训练 60×10^4 次迭代。我们使用权重衰减0.0001和动量0.9。按照 [16]中的做法，我们不使用dropout [14]。

在测试阶段，为便于对比研究，我们采用标准的10-crop测试 [21]。为获得最佳结果，我们采用 [41, 13]，中所述的全卷积形式，并在多个尺度上对分数取平均（图像经过缩放，使短边为 $\{224, 256, 384, 480, 640\}$ ）。

4. 实验

4.1. ImageNet图像分类

我们在ImageNet 2012分类数据集 [36]上评估我们的方法，该数据集包含1000个类别。模型在128万张训练图像上进行训练，并在5万张验证图像上进行评估。我们还在10万张测试图像上获得最终结果，由测试服务器报告。我们评估top-1和top-5错误率。

基础网络。我们首先评估18层和34层的基础网络。34层基础网络如图3（中间）所示。18层基础网络的结构形式类似。详细架构见表1。

表2的结果表明，较深的34层普通网络比较浅的18层普通网络具有更高的验证误差。为了揭示原因，我们在图4（左）中比较了它们在训练过程中的训练/验证误差。我们发现了退化问题——

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
		3×3 max pool, stride 2				
conv2_x	56×56	$\begin{bmatrix} 3\times 3, 64 \\ 3\times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 64 \\ 3\times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3\times 3, 128 \\ 3\times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 128 \\ 3\times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3\times 3, 256 \\ 3\times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 256 \\ 3\times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3\times 3, 512 \\ 3\times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 512 \\ 3\times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10 ⁹	3.6×10 ⁹	3.8×10 ⁹	7.6×10 ⁹	11.3×10 ⁹

Table 1. Architectures for ImageNet. Building blocks are shown in brackets (see also Fig. 5), with the numbers of blocks stacked. Down-sampling is performed by conv3_1, conv4_1, and conv5_1 with a stride of 2.

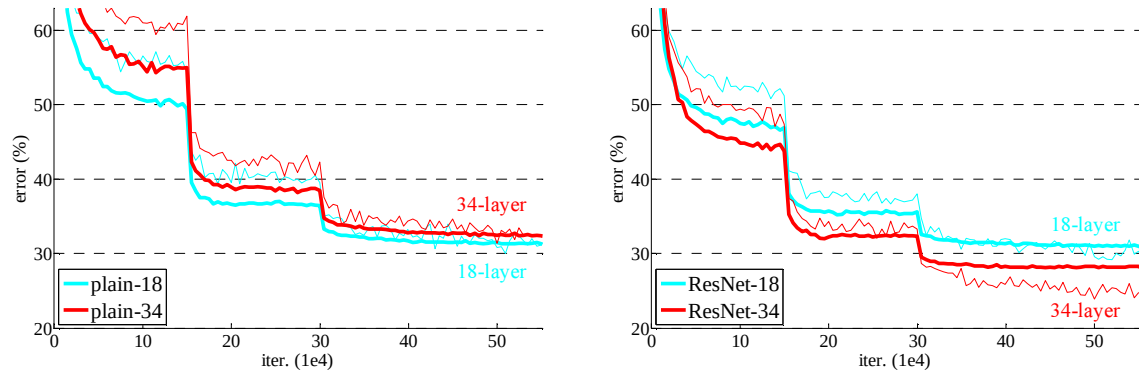


Figure 4. Training on **ImageNet**. Thin curves denote training error, and bold curves denote validation error of the center crops. Left: plain networks of 18 and 34 layers. Right: ResNets of 18 and 34 layers. In this plot, the residual networks have no extra parameter compared to their plain counterparts.

	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	25.03

Table 2. Top-1 error (% , 10-crop testing) on ImageNet validation. Here the ResNets have no extra parameter compared to their plain counterparts. Fig. 4 shows the training procedures.

34-layer plain net has higher *training* error throughout the whole training procedure, even though the solution space of the 18-layer plain network is a subspace of that of the 34-layer one.

We argue that this optimization difficulty is *unlikely* to be caused by vanishing gradients. These plain networks are trained with BN [16], which ensures forward propagated signals to have non-zero variances. We also verify that the backward propagated gradients exhibit healthy norms with BN. So neither forward nor backward signals vanish. In fact, the 34-layer plain net is still able to achieve competitive accuracy (Table 3), suggesting that the solver works to some extent. We conjecture that the deep plain nets may have exponentially low convergence rates, which impact the

reducing of the training error³. The reason for such optimization difficulties will be studied in the future.

Residual Networks. Next we evaluate 18-layer and 34-layer residual nets (*ResNets*). The baseline architectures are the same as the above plain nets, expect that a shortcut connection is added to each pair of 3×3 filters as in Fig. 3 (right). In the first comparison (Table 2 and Fig. 4 right), we use identity mapping for all shortcuts and zero-padding for increasing dimensions (option A). So they have *no extra parameter* compared to the plain counterparts.

We have three major observations from Table 2 and Fig. 4. First, the situation is reversed with residual learning – the 34-layer ResNet is better than the 18-layer ResNet (by 2.8%). More importantly, the 34-layer ResNet exhibits considerably lower training error and is generalizable to the validation data. This indicates that the degradation problem is well addressed in this setting and we manage to obtain accuracy gains from increased depth.

Second, compared to its plain counterpart, the 34-layer

³We have experimented with more training iterations (3×) and still observed the degradation problem, suggesting that this problem cannot be feasibly addressed by simply using more iterations.

层名称 输出尺寸	18层	34层	50层	101层	152层	
conv1	112×112	7×7, 64, 步幅 2				
		3×3最大池化, 步幅 2				
conv2 $\times$	56×56	$\begin{bmatrix} 3\times 3, 64 \\ 3\times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 64 \\ 3\times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$	
conv3 $\times$	28×28	$\begin{bmatrix} 3\times 3, 128 \\ 3\times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 128 \\ 3\times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 8$	
conv4 $\times$	14×14	$\begin{bmatrix} 3\times 3, 256 \\ 3\times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 256 \\ 3\times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 36$	
conv5 $\times$	7×7	$\begin{bmatrix} 3\times 3, 512 \\ 3\times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 512 \\ 3\times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$	
	1×1	平均池化, 1000 维全连接层, softmax				
FLOPs		1.8×10 ⁹	3.6×10 ⁹	3.8×10 ⁹	7.6×10 ⁹	11.3×10 ⁹

表 1. ImageNet 架构。构建块以括号表示（另见图 5），数字表示堆叠的块数。下采样由 conv3_1、conv4_1 和 conv5_1 完成，步幅为 2。

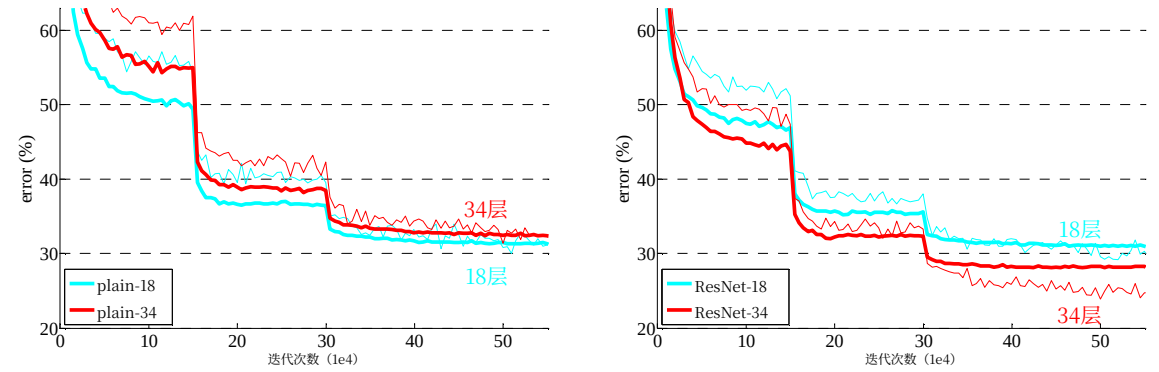


图 4. 在 **ImageNet** 上的训练。细曲线表示训练误差，粗曲线表示中心裁剪的验证误差。左：18 层和 34 层的普通网络。右：18 层和 34 层的残差网络。在本图中，残差网络与其普通对应网络相比没有额外的参数。

	普通网络	残差网络
18 层	27.94	27.88
34层	28.54	25.03

表2. ImageNet验证集上的Top-1错误率（%，10-crop测试）。此处的ResNet与其普通对应网络相比没有额外参数。图4展示了训练过程。

34层普通网络在整个训练过程中始终具有更高的训练误差，尽管18层普通网络的解空间是34层网络解空间的一个子空间。

我们认为这种优化困难是 不太可能 由梯度消失引起的。这些普通网络使用BN [16], 进行训练, BN确保前向传播的信号具有非零方差。我们还验证了在BN下反向传播的梯度具有正常的范数。因此前向和反向信号都没有消失。事实上，34层的普通网络仍然能够达到具有竞争力的准确率（表3），说明求解器在一定程度上是有效的。我们推测深层普通网络可能具有指数级低的收敛率，这会影响

训练误差的减少³。造成此类优化难题的原因将在未来进行研究。

残差网络。接下来我们评估18层和34层的残差网络（*ResNets*）。基线架构与上述普通网络相同，只是在每对 3×3 滤波器之间添加了捷径连接，如图3（右）所示。在第一次比较中（表2和图4右），我们对所有捷径使用恒等映射，并对增加维度的情况使用零填充（选项A）。因此，与普通对应网络相比，它们没有额外参数。

从表2和图4中我们可以得到三个主要观察结果。首先，残差学习逆转了之前的趋势——34层ResNet优于18层ResNet（高出2.8%）。更重要的是，34层ResNet的训练误差显著降低，并且能够很好地泛化到验证数据。这表明在该设置下退化问题得到了很好的解决，我们成功地从增加的深度中获得了准确率提升。

其次，与普通的34层网络相比，

³我们尝试了更多的训练迭代次数（3×），仍然观察到退化问题，这表明简单地增加迭代次数并不能有效地解决该问题。

model	top-1 err.	top-5 err.
VGG-16 [41]	28.07	9.33
GoogLeNet [44]	-	9.15
PReLU-net [13]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	21.43	5.71

Table 3. Error rates (% , **10-crop** testing) on ImageNet validation. VGG-16 is based on our test. ResNet-50/101/152 are of option B that only uses projections for increasing dimensions.

method	top-1 err.	top-5 err.
VGG [41] (ILSVRC’ 14)	-	8.43 [†]
GoogLeNet [44] (ILSVRC’ 14)	-	7.89
VGG [41] (v5)	24.4	7.1
PReLU-net [13]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

Table 4. Error rates (%) of **single-model** results on the ImageNet validation set (except [†] reported on the test set).

method	top-5 err. (test)
VGG [41] (ILSVRC’ 14)	7.32
GoogLeNet [44] (ILSVRC’ 14)	6.66
VGG [41] (v5)	6.8
PReLU-net [13]	4.94
BN-inception [16]	4.82
ResNet (ILSVRC’15)	3.57

Table 5. Error rates (%) of **ensembles**. The top-5 error is on the test set of ImageNet and reported by the test server.

ResNet reduces the top-1 error by 3.5% (Table 2), resulting from the successfully reduced training error (Fig. 4 right *vs.* left). This comparison verifies the effectiveness of residual learning on extremely deep systems.

Last, we also note that the 18-layer plain/residual nets are comparably accurate (Table 2), but the 18-layer ResNet converges faster (Fig. 4 right *vs.* left). When the net is “not overly deep” (18 layers here), the current SGD solver is still able to find good solutions to the plain net. In this case, the ResNet eases the optimization by providing faster convergence at the early stage.

Identity vs. Projection Shortcuts. We have shown that

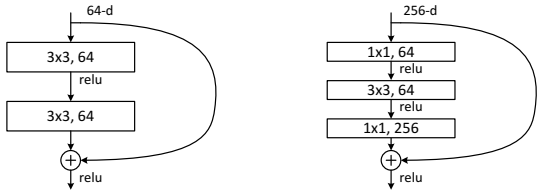


Figure 5. A deeper residual function $\mathcal{F}$ for ImageNet. Left: a building block (on 56×56 feature maps) as in Fig. 3 for ResNet-34. Right: a “bottleneck” building block for ResNet-50/101/152.

parameter-free, identity shortcuts help with training. Next we investigate projection shortcuts (Eqn.(2)). In Table 3 we compare three options: (A) zero-padding shortcuts are used for increasing dimensions, and all shortcuts are parameter-free (the same as Table 2 and Fig. 4 right); (B) projection shortcuts are used for increasing dimensions, and other shortcuts are identity; and (C) all shortcuts are projections.

Table 3 shows that all three options are considerably better than the plain counterpart. B is slightly better than A. We argue that this is because the zero-padded dimensions in A indeed have no residual learning. C is marginally better than B, and we attribute this to the extra parameters introduced by many (thirteen) projection shortcuts. But the small differences among A/B/C indicate that projection shortcuts are not essential for addressing the degradation problem. So we do not use option C in the rest of this paper, to reduce memory/time complexity and model sizes. Identity shortcuts are particularly important for not increasing the complexity of the bottleneck architectures that are introduced below.

Deeper Bottleneck Architectures. Next we describe our deeper nets for ImageNet. Because of concerns on the training time that we can afford, we modify the building block as a *bottleneck* design⁴. For each residual function $\mathcal{F}$, we use a stack of 3 layers instead of 2 (Fig. 5). The three layers are 1×1 , 3×3 , and 1×1 convolutions, where the 1×1 layers are responsible for reducing and then increasing (restoring) dimensions, leaving the 3×3 layer a bottleneck with smaller input/output dimensions. Fig. 5 shows an example, where both designs have similar time complexity.

The parameter-free identity shortcuts are particularly important for the bottleneck architectures. If the identity shortcut in Fig. 5 (right) is replaced with projection, one can show that the time complexity and model size are doubled, as the shortcut is connected to the two high-dimensional ends. So identity shortcuts lead to more efficient models for the bottleneck designs.

50-layer ResNet: We replace each 2-layer block in the

⁴Deeper *non*-bottleneck ResNets (*e.g.*, Fig. 5 left) also gain accuracy from increased depth (as shown on CIFAR-10), but are not as economical as the bottleneck ResNets. So the usage of bottleneck designs is mainly due to practical considerations. We further note that the degradation problem of plain nets is also witnessed for the bottleneck designs.

模型	top-1 错误率	top-5 错误率
VGG-16 [41]	28.07	9.33
GoogLeNet [44]	-	9.15
PReLU-net [13]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	21.43	5.71

表3. ImageNet验证集上的错误率(%, **10裁剪** 测试)。VGG-16 基于我们的测试。ResNet-50/101/152采用选项B, 仅在维度增加时使用投影。

方法	top-1 错误率	top-5 错误率
VGG [41] (ILSVRC’ 14)	-	8.43 [†]
GoogLeNet [44] (ILSVRC’ 14)	-	7.89
VGG [41] (v5)	24.4	7.1
PReLU-net [13]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

表4. **单模型**在 ImageNet 验证集上的错误率(%) ([†] 除外, 其结果报告在测试集上)。

方法	top-5 错误率(测试集)
VGG [41] (ILSVRC’14)	7.32
GoogLeNet [44] (ILSVRC’14)	6.66
VGG [41] (v5)	6.8
PReLU-net [13]	4.94
BN-inception [16]	4.82
ResNet (ILSVRC’ 15)	3.57

表5. **集成**的错误率(%)。top-5错误率是在ImageNet测试集上由测试服务器给出的。

ResNet将top-1错误率降低了3.5% (表2), 这是成功降低训练误差的结果(图4右对比左)。这一比较验证了残差学习在极深网络中的有效性。

最后, 我们还注意到, 18层普通网络/残差网络的准确率相当(表2), 但18层ResNet收敛得更快(图4右对比左)。当网络“不过深”(此处为18层)时, 当前的SGD求解器仍然能够为普通网络找到良好的解。在这种情况下, ResNet通过在早期阶段提供更快收敛来缓解优化问题。

恒等与投影快捷连接。我们已经证明

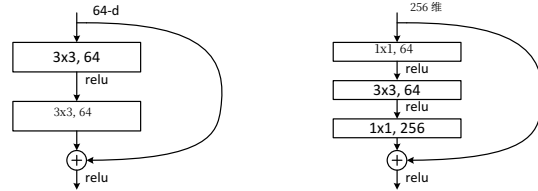


图5. 用于 ImageNet 的更深的残差函数 $\mathcal{F}$ 。左: 与图3中 ResNet-34 相同的构建块(作用于 56×56 特征图)。右: 用于 ResNet-50/101/152 的“瓶颈”构建块。

无参数的恒等快捷连接有助于训练。接下来我们研究投影快捷连接(公式(2))。在表3中我们比较了三种方案: (A) 采用零填充快捷连接来增加维度, 所有快捷连接均无参数(与表2和图4右侧相同); (B) 采用投影快捷连接来增加维度, 其余快捷连接为恒等连接; (C) 所有快捷连接均为投影连接。

表3显示, 所有三个选项都明显优于普通对应版本。B略优于A。我们认为这是因为A中的零填充维度确实没有残差学习。C略优于B, 我们将此归因于许多(十三个)投影快捷连接引入的额外参数。但A/B/C之间的小差异表明, 投影快捷连接对于解决退化问题并非必不可少。因此, 本文其余部分不使用选项C, 以降低内存/时间复杂度和模型大小。恒等快捷连接对于不增加下文引入的瓶颈架构的复杂性尤为重要。

更深的瓶颈架构。接下来, 我们描述用于ImageNet的更深网络。由于担心我们所能承受的训练时间, 我们对构建模块进行了修改, 采用了一种瓶颈设计⁴。对于每个残差函数 $\mathcal{F}$, 我们使用由3层而不是2层组成的堆叠(图5)。这三层分别是 1×1 、 3×3 和 1×1 卷积, 其中 1×1 层负责先降低然后再增加(恢复)维度, 使 3×3 层成为一个输入/输出维度较小的瓶颈层。图5给出了一个示例, 其中两种设计具有相似的时间复杂度。

参数无关的恒等捷径对瓶颈结构尤为重要。如果将图5(右)中的恒等捷径替换为投影, 则可以证明时间复杂度和模型规模都会翻倍, 因为捷径连接到了两端的高维端点。因此, 对于瓶颈设计而言, 恒等捷径能带来更高效的模型。

50层 ResNet: 我们将 34 层网络中的每个两层块替换为

⁴更深的非瓶颈 ResNet (例如图5左) 同样能从深度增加中获得精度提升(如在CIFAR-10上所示), 但其经济性不如瓶颈ResNet。因此, 采用瓶颈设计主要是出于实际考虑。我们进一步注意到, 普通网络的退化问题在瓶颈设计中同样存在。

34-layer net with this 3-layer bottleneck block, resulting in a 50-layer ResNet (Table 1). We use option B for increasing dimensions. This model has 3.8 billion FLOPs.

101-layer and 152-layer ResNets: We construct 101-layer and 152-layer ResNets by using more 3-layer blocks (Table 1). Remarkably, although the depth is significantly increased, the 152-layer ResNet (11.3 billion FLOPs) still has *lower complexity* than VGG-16/19 nets (15.3/19.6 billion FLOPs).

The 50/101/152-layer ResNets are more accurate than the 34-layer ones by considerable margins (Table 3 and 4). We do not observe the degradation problem and thus enjoy significant accuracy gains from considerably increased depth. The benefits of depth are witnessed for all evaluation metrics (Table 3 and 4).

Comparisons with State-of-the-art Methods. In Table 4 we compare with the previous best single-model results. Our baseline 34-layer ResNets have achieved very competitive accuracy. Our 152-layer ResNet has a single-model top-5 validation error of 4.49%. This single-model result outperforms all previous ensemble results (Table 5). We combine six models of different depth to form an ensemble (only with two 152-layer ones at the time of submitting). This leads to **3.57%** top-5 error on the test set (Table 5). *This entry won the 1st place in ILSVRC 2015.*

4.2. CIFAR-10 and Analysis

We conducted more studies on the CIFAR-10 dataset [20], which consists of 50k training images and 10k testing images in 10 classes. We present experiments trained on the training set and evaluated on the test set. Our focus is on the behaviors of extremely deep networks, but not on pushing the state-of-the-art results, so we intentionally use simple architectures as follows.

The plain/residual architectures follow the form in Fig. 3 (middle/right). The network inputs are 32×32 images, with the per-pixel mean subtracted. The first layer is 3×3 convolutions. Then we use a stack of $6n$ layers with 3×3 convolutions on the feature maps of sizes $\{32, 16, 8\}$ respectively, with $2n$ layers for each feature map size. The numbers of filters are $\{16, 32, 64\}$ respectively. The subsampling is performed by convolutions with a stride of 2. The network ends with a global average pooling, a 10-way fully-connected layer, and softmax. There are totally $6n+2$ stacked weighted layers. The following table summarizes the architecture:

output map size	32×32	16×16	8×8
# layers	$1+2n$	$2n$	$2n$
# filters	16	32	64

When shortcut connections are used, they are connected to the pairs of 3×3 layers (totally $3n$ shortcuts). On this dataset we use identity shortcuts in all cases (*i.e.*, option A),

method			error (%)
Maxout [10]			9.38
NIN [25]			8.81
DSN [24]			8.22
	# layers	# params	
FitNet [35]	19	2.5M	8.39
Highway [42, 43]	19	2.3M	7.54 (7.72±0.16)
Highway [42, 43]	32	1.25M	8.80
ResNet	20	0.27M	8.75
ResNet	32	0.46M	7.51
ResNet	44	0.66M	7.17
ResNet	56	0.85M	6.97
ResNet	110	1.7M	6.43 (6.61±0.16)
ResNet	1202	19.4M	7.93

Table 6. Classification error on the **CIFAR-10** test set. All methods are with data augmentation. For ResNet-110, we run it 5 times and show “best (mean±std)” as in [43].

so our residual models have exactly the same depth, width, and number of parameters as the plain counterparts.

We use a weight decay of 0.0001 and momentum of 0.9, and adopt the weight initialization in [13] and BN [16] but with no dropout. These models are trained with a mini-batch size of 128 on two GPUs. We start with a learning rate of 0.1, divide it by 10 at 32k and 48k iterations, and terminate training at 64k iterations, which is determined on a 45k/5k train/val split. We follow the simple data augmentation in [24] for training: 4 pixels are padded on each side, and a 32×32 crop is randomly sampled from the padded image or its horizontal flip. For testing, we only evaluate the single view of the original 32×32 image.

We compare $n = \{3, 5, 7, 9\}$, leading to 20, 32, 44, and 56-layer networks. Fig. 6 (left) shows the behaviors of the plain nets. The deep plain nets suffer from increased depth, and exhibit higher training error when going deeper. This phenomenon is similar to that on ImageNet (Fig. 4, left) and on MNIST (see [42]), suggesting that such an optimization difficulty is a fundamental problem.

Fig. 6 (middle) shows the behaviors of ResNets. Also similar to the ImageNet cases (Fig. 4, right), our ResNets manage to overcome the optimization difficulty and demonstrate accuracy gains when the depth increases.

We further explore $n = 18$ that leads to a 110-layer ResNet. In this case, we find that the initial learning rate of 0.1 is slightly too large to start converging⁵. So we use 0.01 to warm up the training until the training error is below 80% (about 400 iterations), and then go back to 0.1 and continue training. The rest of the learning schedule is as done previously. This 110-layer network converges well (Fig. 6, middle). It has *fewer* parameters than other deep and thin

⁵With an initial learning rate of 0.1, it starts converging (<90% error) after several epochs, but still reaches similar accuracy.

这种三层瓶颈块，从而得到一个 50 层的 ResNet（表 1）。我们使用选项 B 来增加维度。该模型的 FLOPs 为 38 亿。

101层和152层ResNet：我们通过使用更多的3层块来构建101层和152层ResNet（表1）。值得注意的是，尽管深度显著增加，152层ResNet（113亿次浮点运算）的复杂度仍低于VGG-16/19网络（153/196亿次浮点运算）。

50层、101层和152层ResNet比34层ResNet具有显著的精度提升（表3和表4）。我们未观察到退化问题，因此通过显著增加的深度获得了可观的精度收益。深度的益处也在所有评估指标上均有体现（表3和表4）。

与最先进方法的比较。在表4中，我们与此前最佳的单模型结果进行了比较。我们的基线34层ResNet已经达到了非常具有竞争力的精度。我们的152层ResNet单模型top-5验证误差为4.49%。这一单模型结果优于此前所有的集成结果（表5）。我们将六个不同深度的模型组合成一个集成模型（提交时仅包含两个152层模型）。这在测试集上达到了**3.57%**的top-5误差（表5）。该条目在*ILSVRC 2015*中获得第一名。

4.2. CIFAR-10 及分析

我们在 CIFAR-10 数据集[20], 上进行了更多研究，该数据集包含 50k 张训练图像和 10k 张测试图像，共 10 个类别。我们展示了在训练集上训练并在测试集上评估的实验结果。我们的重点在于极深网络的行为，而非追求最先进的结果，因此我们有意使用如下所示的简单架构。

普通/残差架构遵循图 3（中间/右侧）的形式。网络输入为 32×32 图像，减去了逐像素均值。第一层为 3×3 卷积。然后我们使用 $6n$ 层的堆叠，在尺寸分别为 $\{32, 16, 8\}$ 的特征图上进行 3×3 卷积，每个特征图尺寸使用 $2n$ 层。滤波器数量分别为 $\{16, 32, 64\}$ 。子采样通过步长为 2 的卷积实现。网络以全局平均池化、10 类全连接层和 softmax 结束。总共有 $6n+2$ 个堆叠加权层。下表总结了该架构：

输出图尺寸	32×32	16×16	8×8
# 层数	$1+2n$	$2n$	$2n$
# 滤波器数	16	32	64

使用快捷连接时，它们会连接到 3×3 对层之间（共 $3n$ 个快捷连接）。在该数据集上，我们在所有情况下都使用恒等快捷连接（即选项 A），

方法			错误率 (%)
Maxout [10]			9.38
NIN [25]			8.81
DSN [24]			8.22
	层数	参数量	
FitNet [35]	19	2.5M	8.39
Highway [42, 43]	19	2.3M	7.54 (7.72±0.16)
Highway [42, 43]	32	1.25M	8.80
ResNet	20	0.27M	8.75
ResNet	32	0.46M	7.51
ResNet	44	0.66M	7.17
ResNet	56	0.85M	6.97
ResNet	110	1.7M	6.43 (6.61±0.16)
ResNet	1202	19.4M	7.93

表6. 在**CIFAR-10** 测试集上的分类误差。所有方法均采用了数据增强。对于ResNet-110，我们运行了5次，并按 [43]中的方式展示 “best (mean±std)”。

因此我们的残差模型与普通模型具有完全相同的深度、宽度和参数数量。

我们使用权重衰减0.0001和动量0.9，采用 [13] 中的权重初始化和BN [16]，但不使用dropout。这些模型在两块GPU上以128的小批量（mini-batch）大小进行训练。我们从学习率0.1开始，在32k和48k次迭代时将其除以10，并在64k次迭代时终止训练，该训练方案是在45k/5k的训练/验证划分上确定的。训练时我们遵循 [24] 中的简单数据增强方法：在每例填充4个像素，然后从填充后的图像或其水平翻转图像中随机裁剪出一个 32×32 。测试时，我们仅评估原始 32×32 图像的单一视角。

我们对比了 $n = \{3, 5, 7, 9\}$ ，得到20层、32层、44层和56层网络。图6（左）展示了普通网络的表现。深层普通网络因深度增加而受到不利影响，随着深度加深训练误差反而升高。这一现象与在ImageNet（图4，左）和MNIST（见 [42]）上的表现相似，表明这种优化困难是一个根本性问题。

图6（中）展示了ResNet的表现。同样与ImageNet的情况类似（图4，右），我们的ResNet成功克服了优化困难，并在深度增加时展现出精度的提升。

我们进一步探索 $n = 18$ 得到一个110层的ResNet。在这种情况下，我们发现初始学习率0.1略大，不利于开始收敛⁵。因此我们使用0.01来预热训练，直到训练误差低于80%（约400次迭代），然后再恢复到0.1继续训练。其余的学习率调度与之前相同。这个110层网络收敛良好（图6，中间）。它的参数更少，比其他深而窄的

⁵初始学习率为0.1时，经过若干轮训练后开始收敛 (<90%误差)，但最终仍能达到相近的精度。

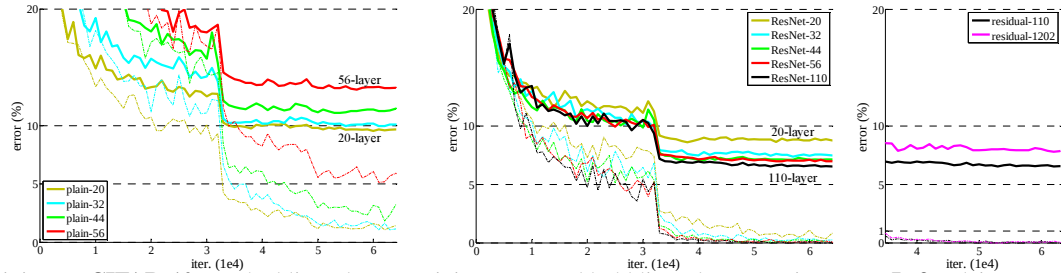


Figure 6. Training on **CIFAR-10**. Dashed lines denote training error, and bold lines denote testing error. **Left**: plain networks. The error of plain-110 is higher than 60% and not displayed. **Middle**: ResNets. **Right**: ResNets with 110 and 1202 layers.

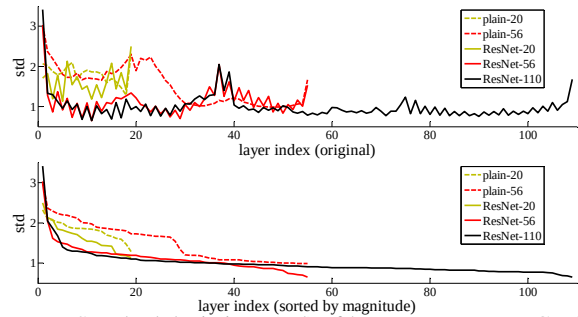


Figure 7. Standard deviations (std) of layer responses on CIFAR-10. The responses are the outputs of each 3×3 layer, after BN and before nonlinearity. **Top**: the layers are shown in their original order. **Bottom**: the responses are ranked in descending order.

networks such as FitNet [35] and Highway [42] (Table 6), yet is among the state-of-the-art results (6.43%, Table 6).

Analysis of Layer Responses. Fig. 7 shows the standard deviations (std) of the layer responses. The responses are the outputs of each 3×3 layer, after BN and before other nonlinearity (ReLU/addition). For ResNets, this analysis reveals the response strength of the residual functions. Fig. 7 shows that ResNets have generally smaller responses than their plain counterparts. These results support our basic motivation (Sec.3.1) that the residual functions might be generally closer to zero than the non-residual functions. We also notice that the deeper ResNet has smaller magnitudes of responses, as evidenced by the comparisons among ResNet-20, 56, and 110 in Fig. 7. When there are more layers, an individual layer of ResNets tends to modify the signal less.

Exploring Over 1000 layers. We explore an aggressively deep model of over 1000 layers. We set $n = 200$ that leads to a 1202-layer network, which is trained as described above. Our method shows *no optimization difficulty*, and this 10^3 -layer network is able to achieve *training error* $< 0.1\%$ (Fig. 6, right). Its test error is still fairly good (7.93%, Table 6).

But there are still open problems on such aggressively deep models. The testing result of this 1202-layer network is worse than that of our 110-layer network, although both

training data	07+12	07++12
test data	VOC 07 test	VOC 12 test
VGG-16	73.2	70.4
ResNet-101	76.4	73.8

Table 7. Object detection mAP (%) on the PASCAL VOC 2007/2012 test sets using **baseline** Faster R-CNN. See also Table 10 and 11 for better results.

metric	mAP@.5	mAP@[.5, .95]
VGG-16	41.5	21.2
ResNet-101	48.4	27.2

Table 8. Object detection mAP (%) on the COCO validation set using **baseline** Faster R-CNN. See also Table 9 for better results.

have similar training error. We argue that this is because of overfitting. The 1202-layer network may be unnecessarily large (19.4M) for this small dataset. Strong regularization such as maxout [10] or dropout [14] is applied to obtain the best results ([10, 25, 24, 35]) on this dataset. In this paper, we use no maxout/dropout and just simply impose regularization via deep and thin architectures by design, without distracting from the focus on the difficulties of optimization. But combining with stronger regularization may improve results, which we will study in the future.

4.3. Object Detection on PASCAL and MS COCO

Our method has good generalization performance on other recognition tasks. Table 7 and 8 show the object detection baseline results on PASCAL VOC 2007 and 2012 [5] and COCO [26]. We adopt *Faster R-CNN* [32] as the detection method. Here we are interested in the improvements of replacing VGG-16 [41] with ResNet-101. The detection implementation (see appendix) of using both models is the same, so the gains can only be attributed to better networks. Most remarkably, on the challenging COCO dataset we obtain a 6.0% increase in COCO’s standard metric (mAP@[.5, .95]), which is a 28% relative improvement. This gain is solely due to the learned representations.

Based on deep residual nets, we won the 1st places in several tracks in ILSVRC & COCO 2015 competitions: ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation. The details are in the appendix.

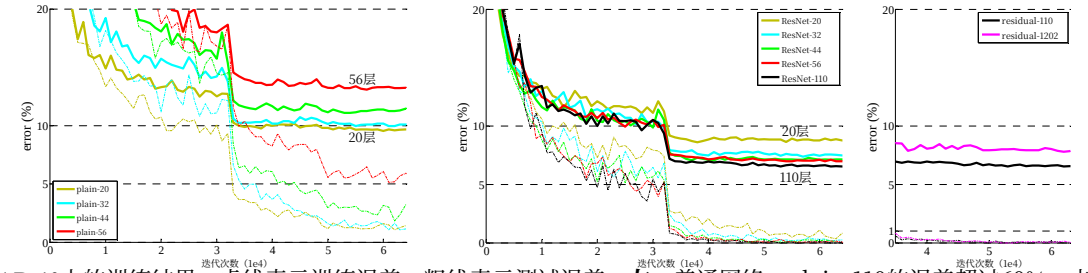


图6. 在**CIFAR-10**上的训练结果。虚线表示训练误差，粗线表示测试误差。左：普通网络。plain-110的误差超过60%，未在图中显示。中：ResNets。右：110层和1202层的ResNets。

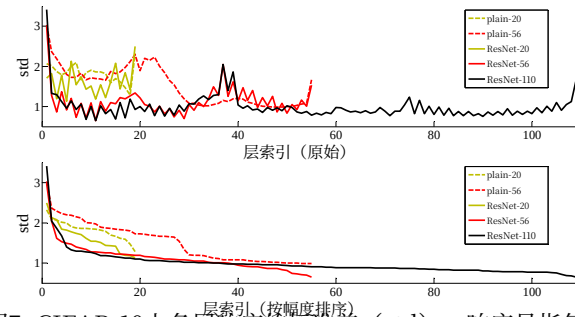


图7. CIFAR-10上各层响应的标准差 (std)。响应是指每个 3×3 层在BN之后、非线性变换之前的输出。上：按各层原始顺序显示。下：响应按降序排列。

网络（如FitNet [35]和Highway [42]，表6）更少，但仍处于最先进的结果之列（6.43%，表6）。

层响应分析。图7显示了各层响应的标准差 (std)。响应是指每个 3×3 层在BN之后、其他非线性变换（ReLU/加法）之前的输出。对于ResNets，该分析揭示了残差函数的响应强度。图7表明，ResNets的响应通常比其对应的普通网络更小。这些结果支持我们的基本动机（3.1节），即残差函数可能比非残差函数普遍更接近于零。我们还注意到，层数越深的ResNet其响应幅度越小，图7中ResNet-20、56和110的对比也证实了这一点。当层数更多时，ResNets中的单个层倾向于对信号的修改更少。

探索超过1000层的网络我们探索了一个超过1000层的激进深度模型。我们设置 $n = 200$ ，由此得到一个1202层的网络，按上述方式训练。我们的方法没有显示出任何优化困难，且这个 10^3 层网络能够达到训练误差 $< 0.1\%$ （图6，右）。其测试误差仍然相当好（7.93%，表6）。

然而，这种激进的深度模型仍存在未解决的问题。这个1202层网络的测试结果比我们110层网络的结果更差，尽管两者

训练数据	07+12	07++12
测试数据	VOC 07 测试集	VOC 12 测试集
VGG-16	73.2	70.4
ResNet-101	76.4	73.8

表 7. 使用**基线** Faster R-CNN 在 PASCAL VOC 2007/2012 测试集上的目标检测 mAP (%)。更好的结果另见表 10 和表 11。

指标	mAP@.5	mAP@[.5, .95]
VGG-16	41.5	21.2
ResNet-101	48.4	27.2

表 8. 使用**基线** Faster R-CNN在COCO验证集上的目标检测 mAP (%)。更好的结果请参见表9。

的训练误差相近。我们认为这是过拟合所致。对于这个小型数据集，1202层网络可能过于庞大（19.4M）。在该数据集上，通常会采用如maxout [10]或dropout [14]等强正则化方法来获得最佳结果（[10, 25, 24, 35]）。在本文中，我们不使用maxout/dropout，而是通过精心设计的深而窄的架构简单地施加正则化，以免偏离对优化难题的关注。但结合更强的正则化可能会改善结果，我们将在未来进行研究。

4.3. PASCAL和MS COCO上的目标检测

我们的方法在其他识别任务上具有良好的泛化性能。表7和表8分别展示了在PASCAL VOC 2007和2012[5]以及COCO [26]上的目标检测基线结果。我们采用*Faster R-CNN* [32]作为检测方法。这里我们关注的是将VGG-16 [41]替换为ResNet-101所带来的提升。两种模型的检测实现（参见附录）是相同的，因此性能提升只能归因于更好的网络结构。最引人注目的是，在具有挑战性的COCO数据集上，我们在COCO的标准指标（mAP@[.5, .95]）上获得了6.0%的提升，相对提升达28%。这一提升完全归功于所学习到的表示能力。

基于深度残差网络，我们在ILSVRC & COCO 2015竞赛的多个赛道中获得了第一名：ImageNet检测、ImageNet定位、COCO检测以及COCO分割。详细信息见附录。

References

- [1] Y. Bengio, P. Simard, and P. Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994.
- [2] C. M. Bishop. *Neural networks for pattern recognition*. Oxford university press, 1995.
- [3] W. L. Briggs, S. F. McCormick, et al. *A Multigrid Tutorial*. Siam, 2000.
- [4] K. Chatfield, V. Lempitsky, A. Vedaldi, and A. Zisserman. The devil is in the details: an evaluation of recent feature encoding methods. In *BMVC*, 2011.
- [5] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman. The Pascal Visual Object Classes (VOC) Challenge. *IJCV*, pages 303–338, 2010.
- [6] S. Gidaris and N. Komodakis. Object detection via a multi-region & semantic segmentation-aware cnn model. In *ICCV*, 2015.
- [7] R. Girshick. Fast R-CNN. In *ICCV*, 2015.
- [8] R. Girshick, J. Donahue, T. Darrell, and J. Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *CVPR*, 2014.
- [9] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *AISTATS*, 2010.
- [10] I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, and Y. Bengio. Maxout networks. *arXiv:1302.4389*, 2013.
- [11] K. He and J. Sun. Convolutional neural networks at constrained time cost. In *CVPR*, 2015.
- [12] K. He, X. Zhang, S. Ren, and J. Sun. Spatial pyramid pooling in deep convolutional networks for visual recognition. In *ECCV*, 2014.
- [13] K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *ICCV*, 2015.
- [14] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv:1207.0580*, 2012.
- [15] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [16] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015.
- [17] H. Jegou, M. Douze, and C. Schmid. Product quantization for nearest neighbor search. *TPAMI*, 33, 2011.
- [18] H. Jegou, F. Perronnin, M. Douze, J. Sanchez, P. Perez, and C. Schmid. Aggregating local image descriptors into compact codes. *TPAMI*, 2012.
- [19] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell. Caffe: Convolutional architecture for fast feature embedding. *arXiv:1408.5093*, 2014.
- [20] A. Krizhevsky. Learning multiple layers of features from tiny images. *Tech Report*, 2009.
- [21] A. Krizhevsky, I. Sutskever, and G. Hinton. Imagenet classification with deep convolutional neural networks. In *NIPS*, 2012.
- [22] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1989.
- [23] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller. Efficient backprop. In *Neural Networks: Tricks of the Trade*, pages 9–50. Springer, 1998.
- [24] C.-Y. Lee, S. Xie, P. Gallagher, Z. Zhang, and Z. Tu. Deeply-supervised nets. *arXiv:1409.5185*, 2014.
- [25] M. Lin, Q. Chen, and S. Yan. Network in network. *arXiv:1312.4400*, 2013.
- [26] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common objects in context. In *ECCV*. 2014.
- [27] J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. In *CVPR*, 2015.

- [28] G. Montúfar, R. Pascanu, K. Cho, and Y. Bengio. On the number of linear regions of deep neural networks. In *NIPS*, 2014.
- [29] V. Nair and G. E. Hinton. Rectified linear units improve restricted boltzmann machines. In *ICML*, 2010.
- [30] F. Perronnin and C. Dance. Fisher kernels on visual vocabularies for image categorization. In *CVPR*, 2007.
- [31] T. Raiko, H. Valpola, and Y. LeCun. Deep learning made easier by linear transformations in perceptrons. In *AISTATS*, 2012.
- [32] S. Ren, K. He, R. Girshick, and J. Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *NIPS*, 2015.
- [33] S. Ren, K. He, R. Girshick, X. Zhang, and J. Sun. Object detection networks on convolutional feature maps. *arXiv:1504.06066*, 2015.
- [34] B. D. Ripley. *Pattern recognition and neural networks*. Cambridge university press, 1996.
- [35] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio. Fitnets: Hints for thin deep nets. In *ICLR*, 2015.
- [36] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, et al. Imagenet large scale visual recognition challenge. *arXiv:1409.0575*, 2014.
- [37] A. M. Saxe, J. L. McClelland, and S. Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. *arXiv:1312.6120*, 2013.
- [38] N. N. Schraudolph. Accelerated gradient descent by factor-centering decomposition. Technical report, 1998.
- [39] N. N. Schraudolph. Centering neural network gradient factors. In *Neural Networks: Tricks of the Trade*, pages 207–226. Springer, 1998.
- [40] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. LeCun. Overfeat: Integrated recognition, localization and detection using convolutional networks. In *ICLR*, 2014.
- [41] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015.
- [42] R. K. Srivastava, K. Greff, and J. Schmidhuber. Highway networks. *arXiv:1505.00387*, 2015.
- [43] R. K. Srivastava, K. Greff, and J. Schmidhuber. Training very deep networks. *1507.06228*, 2015.
- [44] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. In *CVPR*, 2015.
- [45] R. Szeliski. Fast surface interpolation using hierarchical basis functions. *TPAMI*, 1990.
- [46] R. Szeliski. Locally adapted hierarchical basis preconditioning. In *SIGGRAPH*, 2006.
- [47] T. Vatanen, T. Raiko, H. Valpola, and Y. LeCun. Pushing stochastic gradient towards second-order methods—backpropagation learning with transformations in nonlinearities. In *Neural Information Processing*, 2013.
- [48] A. Vedaldi and B. Fulkerson. VLFeat: An open and portable library of computer vision algorithms, 2008.
- [49] W. Venables and B. Ripley. Modern applied statistics with s-plus. 1999.
- [50] M. D. Zeiler and R. Fergus. Visualizing and understanding convolutional neural networks. In *ECCV*, 2014.

参考文献

- [1] Y. Bengio, P. Simard, 和 P. Frasconi. 用梯度下降学习长期依赖很困难. IEEE Transactions on Neural Networks, 5(2):157–166, 1994.[2] C. M. Bishop. 用于模式识别的神经网络. 牛津大学出版社, 1995.[3] W. L. Briggs, S. F. McCormick, 等. 多重网格教程. Siam, 2000.[4] K. Chatfield, V. Lempitsky, A. Vedaldi, 和 A. Zisserman. 细节决定成败：近期特征编码方法的评价. 收录于 BMVC, 2011.[5] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, 和 A. Zisserman. Pascal 视觉目标类（VOC）挑战赛. IJCV, 第 303–338 页, 2010.[6] S. Gidaris 和 N. Komodakis. 通过多区域与语义分割感知的 CNN 模型进行目标检测. 收录于 ICCV, 2015.[7] R. Girshick. Fast R-CNN. 收录于 ICCV, 2015.[8] R. Girshick, J. Donahue, T. Darrell, 和 J. Malik. 用于精确目标检测和语义分割的丰富特征层次结构. 收录于 CVPR, 2014.
- [9] X. Glorot 和 Y. Bengio. 理解训练深层前馈神经网络的难度. 收录于 AISTATS, 2010.[10] I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, 和 Y. Bengio. Maxout 网络. arXiv:1302.4389, 2013.[11] K. He 和 J. Sun. 受限时间开销下的卷积神经网络. 收录于 CVPR, 2015.[12] K. He, X. Zhang, S. Ren, 和 J. Sun. 用于视觉识别的深度卷积网络中的空间金字塔池化. 收录于 ECCV, 2014.[13] K. He, X. Zhang, S. Ren, 和 J. Sun. 深入研究整流器：在 ImageNet 分类上超越人类水平的性能. 收录于 ICCV, 2015.[14] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, 和 R. R. Salakhutdinov. 通过防止特征检测器的协同适应来改进神经网络. arXiv:1207.0580, 2012.[15] S. Hochreiter 和 J. Schmidhuber. 长短期记忆. Neural computation, 9(8):1735–1780, 1997.[16] S. Ioffe 和 C. Szegedy. 批归一化：通过减少内部协方差偏移来加速深度网络训练. 收录于 ICML, 2015.[17] H. Jegou, M. Douze, 和 C. Schmid. 用于最近邻搜索的乘积量化. TPAMI, 33, 2011.[18] H. Jegou, F. Perronnin, M. Douze, J. Sanchez, P. Perez, 和 C. Schmid. 将局部图像描述子聚合为紧凑编码. TPAMI, 2012.[19] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, 和 T. Darrell. Caffe: Convolutional architecture for fast feature embedding. arXiv:1408.5093, 2014.[20] A. Krizhevsky. 学习多个特征层从微小图像. arXiv:1408.5093, 2014.[21] A. Krizhevsky, I. Sutskever, 和 G. Hinton. 基于深度卷积神经网络的 ImageNet 分类. 收录于 NIPS, 2012.[22] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, 和 L. D. Jackel. 反向传播应用于手写邮政编码识别. Neural computation, 1989.[23] Y. LeCun, L. Bottou, G. B. Orr, 和 K.-R. Müller. 高效反向传播. 收录于 Neural Networks: Tricks of the Trade, 第 9–50 页. Springer, 1998.[24] C.-Y. Lee, S. Xie, P. Gallagher, Z. Zhang, 和 Z. Tu. 深度监督网络. arXiv:1409.5185, 2014.[25] M. Lin, Q. Chen, 和 S. Yan. 网络中的网络. arXiv:1312.4400, 2013.[26] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, 和 C. L. Zitnick. Microsoft COCO：上下文中的常见目标. 收录于 ECCV, 2014.[27] J. Long, E. Shelhamer, 和 T. Darrell. 用于语义分割的全卷积网络. 收录于 CVPR, 2015.

- [28] G. Montúfar, R. Pascanu, K. Cho, and Y. Bengio. On the number of linear regions of deep neural networks. 收录于 NIPS, 2014.[29] V. Nair and G. E. Hinton. Rectified linear units improve restricted boltzmann machines. 收录于 ICML, 2010.[30] F. Perronnin and C. Dance. Fisher kernels on visual vocabularies for image categorization. 收录于 CVPR, 2007.[31] T. Raiko, H. Valpola, and Y. LeCun. Deep learning made easier by linear transformations in perceptrons. 收录于 AISTATS, 2012.[32] S. Ren, K. He, R. Girshick, and J. Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. 收录于 NIPS, 2015.
- [33] S. Ren, K. He, R. Girshick, X. Zhang, and J. Sun. Object detection networks on convolutional feature maps. arXiv:1504.06066, 2015.[34] B. D. Ripley. Pattern recognition and neural networks. 剑桥大学出版社, 1996.[35] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio. Fitnets: Hints for thin deep nets. 收录于 ICLR, 2015.[36] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, et al. Imagenet large scale visual recognition challenge. arXiv:1409.0575, 2014.[37] A. M. Saxe, J. L. McClelland, and S. Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. arXiv:1312.6120, 2013.[38] N. N. Schraudolph. Accelerated gradient descent by factor-centering decomposition. 技术报告, 1998.[39] N. N. Schraudolph. Centering neural network gradient factors. 收录于 Neural Networks: Tricks of the Trade, 第207–226页. Springer, 1998.
- [40] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. LeCun. Overfeat: Integrated recognition, localization and detection using convolutional networks. 收录于 ICLR, 2014.[41] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. 收录于 ICLR, 2015.[42] R. K. Srivastava, K. Greff, and J. Schmidhuber. Highway networks. arXiv:1505.00387, 2015.[43] R. K. Srivastava, K. Greff, and J. Schmidhuber. Training very deep networks. 1507.06228, 2015.[44] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. 收录于 CVPR, 2015.[45] R. Szeliski. Fast surface interpolation using hierarchical basis functions. TPAMI, 1990.[46] R. Szeliski. Locally adapted hierarchical basis preconditioning. 收录于 SIGGRAPH, 2006.[47] T. Vatanen, T. Raiko, H. Valpola, and Y. LeCun. Pushing stochastic gradient towards second-order methods—backpropagation learning with transformations in nonlinearities. 收录于 Neural Information Processing, 2013.[48] A. Vedaldi and B. Fulkerson. VLFeat: An open and portable library of computer vision algorithms, 2008.[49] W. Venables and B. Ripley. Modern applied statistics with s-plus. 1999.[50] M. D. Zeiler and R. Fergus. Visualizing and understanding convolutional neural networks. 收录于 ECCV, 2014.

A. Object Detection Baselines

In this section we introduce our detection method based on the baseline Faster R-CNN [32] system. The models are initialized by the ImageNet classification models, and then fine-tuned on the object detection data. We have experimented with ResNet-50/101 at the time of the ILSVRC & COCO 2015 detection competitions.

Unlike VGG-16 used in [32], our ResNet has no hidden fc layers. We adopt the idea of “Networks on Conv feature maps” (NoC) [33] to address this issue. We compute the full-image shared conv feature maps using those layers whose strides on the image are no greater than 16 pixels (*i.e.*, conv1, conv2_x, conv3_x, and conv4_x, totally 91 conv layers in ResNet-101; Table 1). We consider these layers as analogous to the 13 conv layers in VGG-16, and by doing so, both ResNet and VGG-16 have conv feature maps of the same total stride (16 pixels). These layers are shared by a region proposal network (RPN, generating 300 proposals) [32] and a Fast R-CNN detection network [7]. RoI pooling [7] is performed before conv5_1. On this RoI-pooled feature, all layers of conv5_x and up are adopted for each region, playing the roles of VGG-16’s fc layers. The final classification layer is replaced by two sibling layers (classification and box regression [7]).

For the usage of BN layers, after pre-training, we compute the BN statistics (means and variances) for each layer on the ImageNet training set. Then the BN layers are fixed during fine-tuning for object detection. As such, the BN layers become linear activations with constant offsets and scales, and BN statistics are not updated by fine-tuning. We fix the BN layers mainly for reducing memory consumption in Faster R-CNN training.

PASCAL VOC

Following [7, 32], for the PASCAL VOC 2007 *test* set, we use the 5k *trainval* images in VOC 2007 and 16k *trainval* images in VOC 2012 for training (“07+12”). For the PASCAL VOC 2012 *test* set, we use the 10k *trainval+test* images in VOC 2007 and 16k *trainval* images in VOC 2012 for training (“07++12”). The hyper-parameters for training Faster R-CNN are the same as in [32]. Table 7 shows the results. ResNet-101 improves the mAP by >3% over VGG-16. This gain is solely because of the improved features learned by ResNet.

MS COCO

The MS COCO dataset [26] involves 80 object categories. We evaluate the PASCAL VOC metric (mAP @ IoU = 0.5) and the standard COCO metric (mAP @ IoU = .5:.05:.95). We use the 80k images on the train set for training and the 40k images on the val set for evaluation. Our detection system for COCO is similar to that for PASCAL VOC. We train the COCO models with an 8-GPU implementation, and thus the RPN step has a mini-batch size of

8 images (*i.e.*, 1 per GPU) and the Fast R-CNN step has a mini-batch size of 16 images. The RPN step and Fast R-CNN step are both trained for 240k iterations with a learning rate of 0.001 and then for 80k iterations with 0.0001.

Table 8 shows the results on the MS COCO validation set. ResNet-101 has a 6% increase of mAP@[.5, .95] over VGG-16, which is a 28% relative improvement, solely contributed by the features learned by the better network. Remarkably, the mAP@[.5, .95]’s absolute increase (6.0%) is nearly as big as mAP@.5’s (6.9%). This suggests that a deeper network can improve both recognition and localization.

B. Object Detection Improvements

For completeness, we report the improvements made for the competitions. These improvements are based on deep features and thus should benefit from residual learning.

MS COCO

Box refinement. Our box refinement partially follows the iterative localization in [6]. In Faster R-CNN, the final output is a regressed box that is different from its proposal box. So for inference, we pool a new feature from the regressed box and obtain a new classification score and a new regressed box. We combine these 300 new predictions with the original 300 predictions. Non-maximum suppression (NMS) is applied on the union set of predicted boxes using an IoU threshold of 0.3 [8], followed by box voting [6]. Box refinement improves mAP by about 2 points (Table 9).

Global context. We combine global context in the Fast R-CNN step. Given the full-image conv feature map, we pool a feature by global Spatial Pyramid Pooling [12] (with a “single-level” pyramid) which can be implemented as “RoI” pooling using the entire image’s bounding box as the RoI. This pooled feature is fed into the post-RoI layers to obtain a global context feature. This global feature is concatenated with the original per-region feature, followed by the sibling classification and box regression layers. This new structure is trained end-to-end. Global context improves mAP@.5 by about 1 point (Table 9).

Multi-scale testing. In the above, all results are obtained by single-scale training/testing as in [32], where the image’s shorter side is $s = 600$ pixels. Multi-scale training/testing has been developed in [12, 7] by selecting a scale from a feature pyramid, and in [33] by using maxout layers. In our current implementation, we have performed multi-scale *testing* following [33]; we have not performed multi-scale training because of limited time. In addition, we have performed multi-scale testing only for the Fast R-CNN step (but not yet for the RPN step). With a trained model, we compute conv feature maps on an image pyramid, where the image’s shorter sides are $s \in \{200, 400, 600, 800, 1000\}$.

A. 目标检测基线

在本节中，我们介绍基于基线Faster R-CNN [32]系统的检测方法。模型先由ImageNet分类模型初始化，然后在目标检测数据上进行微调。我们在ILSVRC与COCO 2015目标检测竞赛期间，对ResNet-50/101进行了实验。

与 [32], 中使用的VGG-16不同，我们的ResNet没有隐藏的全连接层。我们采用了"基于卷积特征图的网络" (NoC) [33] 的思想来解决这个问题。我们使用那些在图像上步长不超过16像素的层来计算全图像共享的卷积特征图（即conv1、conv2_x、conv3_x和conv4_x，ResNet-101中共91个卷积层；表1）。我们将这些层视为与VGG-16中的13个卷积层相对应的层，通过这种方式，ResNet和VGG-16都具有相同总步长（16像素）的卷积特征图。这些层被区域提议网络（RPN，生成300个提议）[32] 和Fast R-CNN检测网络 [7]共享。RoI池化 [7] 在conv5_1之前执行。在该RoI池化特征上，对每个区域采用conv5_x及其之后的所有层，起到VGG-16中全连接层的作用。最终的分类层被替换为两个并行层（分类和边界框回归 [7]）。

关于BN层的使用，预训练之后，我们在ImageNet训练集上计算每一层的BN统计量（均值和方差）。然后在目标检测的微调过程中固定BN层。这样，BN层就变成了具有固定偏移和缩放的线性激活，BN统计量也不会被微调更新。我们固定BN层主要是为了减少Faster R-CNN训练时的内存消耗。

PASCAL VOC

Following [7, 32], for the PASCAL VOC 2007 *test* set, we use the 5k *trainval* images in VOC 2007 and 16k *trainval* images in VOC 2012 for training (“07+12”). For the PASCAL VOC 2012 *test* set, we use the 10k *trainval+test* images in VOC 2007 and 16k *trainval* images in VOC 2012 for training (“07++12”). The hyper-parameters for training Faster R-CNN are the same as in [32]. Table 7 shows the results. ResNet-101 improves the mAP by >3% over VGG-16. This gain is solely because of the improved features learned by ResNet.

MS COCO

MS COCO 数据集 [26] 包含 80 个目标类别。我们采用 PASCAL VOC 评估指标（mAP@IoU = 0.5）和标准 COCO 评估指标（mAP @IoU =.5:.05:.95）。我们使用训练集中的 80k 张图像进行训练，使用验证集中的 40k 张图像进行评估。我们在 COCO 上的检测系统与 PASCALVOC 上的类似。我们使用 8 GPU 实现来训练 COCO 模型，因此 RPN 阶段的小批量大小为

8 张图像（即，每 GPU 1 张），Fast R-CNN 阶段的小批量大小为 16 张图像。RPN 阶段和 Fast R-CNN 阶段均以 0.001 的学习率训练 240k 次迭代，然后以 0.0001 的学习率训练 80k 次迭代。

表8展示了在MS COCO验证集上的结果。ResNet-101相比VGG-16在mAP@[.5, .95]上提升了6%，即相对提升了28%，这完全得益于更优网络所学到的特征。值得注意的是，mAP@[.5, .95]的绝对增幅（6.0%）几乎与mAP@.5的增幅（6.9%）相当。这表明更深的网络可以同时提升识别和定位能力。

B. 目标检测改进

为完整起见，我们报告了竞赛中所作的改进。这些改进基于深度特征，因此应当受益于残差学习。

MS COCO

边界框细化。我们的边界框细化部分遵循 [6]中的迭代定位方法。在Faster R-CNN中，最终输出是一个回归后的边界框，与原始建议框不同。因此在推理时，我们从回归后的边界框中池化新的特征，获得新的分类得分和新的回归边界框。我们将这300个新预测与原始的300个预测合并。对预测框的并集采用非极大值抑制（NMS），IoU阈值设为0.3 [8]，随后进行边界框投票 [6]。边界框细化使mAP提升了约2个百分点（表9）。

全局上下文。我们在 Fast R-CNN 步骤中融合全局上下文。给定整幅图像的卷积特征图，我们通过全局空间金字塔池化 [12]（采用"单层级"金字塔）来池化特征，该池化可以实现为以整幅图像的边界框作为 RoI 的"RoI"池化。该池化特征被送入 RoI 后的各层以获得全局上下文特征。该全局特征与原始的逐区域特征进行拼接，随后送入同级的分类和边界框回归层。这一新结构以端到端方式进行训练。全局上下文将 mAP@.5 提升了约 1 个百分点（表 9）。

多尺度测试。在上述实验中，所有结果均通过单尺度训练/测试获得，如 [32], 所示，其中图像的较短边为 $s = 600$ 像素。多尺度训练/测试已在 [12, 7] 中通过从特征金字塔中选择尺度来实现，并在 [33] 中通过使用 maxout 层来实现。在我们的当前实现中，我们按照 [33] 进行了多尺度测试；由于时间有限，我们未进行多尺度训练。此外，我们仅对 Fast R-CNN 步骤进行了多尺度测试（尚未对 RPN 步骤进行多尺度测试）。对于训练好的模型，我们在图像金字塔上计算卷积特征图，其中图像的较短边分别为 $s \in \{200, 400, 600, 800, 1000\}$ 。

training data	COCO train		COCO trainval	
test data	COCO val		COCO test-dev	
mAP	@.5	@[.5, .95]	@.5	@[.5, .95]
baseline Faster R-CNN (VGG-16)	41.5	21.2		
baseline Faster R-CNN (ResNet-101)	48.4	27.2		
+box refinement	49.9	29.9		
+context	51.1	30.0	53.3	32.2
+multi-scale testing	53.8	32.5	55.7	34.9
ensemble			59.0	37.4

Table 9. Object detection improvements on MS COCO using Faster R-CNN and ResNet-101.

system	net	data	mAP	areo	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
baseline	VGG-16	07+12	73.2	76.5	79.0	70.9	65.5	52.1	83.1	84.7	86.4	52.0	81.9	65.7	84.8	84.6	77.5	76.7	38.8	73.6	73.9	83.0	72.6
baseline	ResNet-101	07+12	76.4	79.8	80.7	76.2	68.3	55.9	85.1	85.3	89.8	56.7	87.8	69.4	88.3	88.9	80.9	78.4	41.7	78.6	79.8	85.3	72.0
baseline+++	ResNet-101	COCO+07+12	85.6	90.0	89.6	87.8	80.8	76.1	89.9	89.9	89.6	75.5	90.0	80.7	89.6	90.3	89.1	88.7	65.4	88.1	85.6	89.0	86.8

Table 10. Detection results on the PASCAL VOC 2007 test set. The baseline is the Faster R-CNN system. The system “baseline+++” include box refinement, context, and multi-scale testing in Table 9.

system	net	data	mAP	areo	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
baseline	VGG-16	07++12	70.4	84.9	79.8	74.3	53.9	49.8	77.5	75.9	88.5	45.6	77.1	55.3	86.9	81.7	80.9	79.6	40.1	72.6	60.9	81.2	61.5
baseline	ResNet-101	07++12	73.8	86.5	81.6	77.2	58.0	51.0	78.6	76.6	93.2	48.6	80.4	59.0	92.1	85.3	84.8	80.7	48.1	77.3	66.5	84.7	65.6
baseline+++	ResNet-101	COCO+07++12	83.8	92.1	88.4	84.8	75.9	71.4	86.3	87.8	94.2	66.8	89.4	69.2	93.9	91.9	90.9	89.6	67.9	88.2	76.8	90.3	80.0

Table 11. Detection results on the PASCAL VOC 2012 test set (<http://host.robots.ox.ac.uk:8080/leaderboard/displaylb.php?challengeid=11&compid=4>). The baseline is the Faster R-CNN system. The system “baseline+++” include box refinement, context, and multi-scale testing in Table 9.

We select two adjacent scales from the pyramid following [33]. RoI pooling and subsequent layers are performed on the feature maps of these two scales [33], which are merged by maxout as in [33]. Multi-scale testing improves the mAP by over 2 points (Table 9).

Using validation data. Next we use the 80k+40k trainval set for training and the 20k test-dev set for evaluation. The test-dev set has no publicly available ground truth and the result is reported by the evaluation server. Under this setting, the results are an mAP@.5 of 55.7% and an mAP@[.5, .95] of 34.9% (Table 9). This is our single-model result.

Ensemble. In Faster R-CNN, the system is designed to learn region proposals and also object classifiers, so an ensemble can be used to boost both tasks. We use an ensemble for proposing regions, and the union set of proposals are processed by an ensemble of per-region classifiers. Table 9 shows our result based on an ensemble of 3 networks. The mAP is 59.0% and 37.4% on the test-dev set. *This result won the 1st place in the detection task in COCO 2015.*

PASCAL VOC

We revisit the PASCAL VOC dataset based on the above model. With the single model on the COCO dataset (55.7% mAP@.5 in Table 9), we fine-tune this model on the PASCAL VOC sets. The improvements of box refinement, context, and multi-scale testing are also adopted. By doing so

	val2	test
GoogLeNet [44] (ILSVRC’14)	-	43.9
our single model (ILSVRC’15)	60.5	58.8
our ensemble (ILSVRC’15)	63.6	62.1

Table 12. Our results (mAP, %) on the ImageNet detection dataset. Our detection system is Faster R-CNN [32] with the improvements in Table 9, using ResNet-101.

we achieve 85.6% mAP on PASCAL VOC 2007 (Table 10) and 83.8% on PASCAL VOC 2012 (Table 11)⁶. The result on PASCAL VOC 2012 is 10 points higher than the previous state-of-the-art result [6].

ImageNet Detection

The ImageNet Detection (DET) task involves 200 object categories. The accuracy is evaluated by mAP@.5. Our object detection algorithm for ImageNet DET is the same as that for MS COCO in Table 9. The networks are pre-trained on the 1000-class ImageNet classification set, and are fine-tuned on the DET data. We split the validation set into two parts (val1/val2) following [8]. We fine-tune the detection models using the DET training set and the val1 set. The val2 set is used for validation. We do not use other ILSVRC 2015 data. Our single model with ResNet-101 has

⁶<http://host.robots.ox.ac.uk:8080/anonymous/30J40J.html>, submitted on 2015-11-26.

训练数据	COCO train		COCO trainval	
测试数据	COCO val		COCO test-dev	
mAP	@.5	@[.5, .95]	@.5	@[.5, .95]
基线 Faster R-CNN (VGG-16)	41.5	21.2		
基线 Faster R-CNN (ResNet-101)	48.4	27.2		
+边界框细化	49.9	29.9		
+上下文	51.1	30.0	53.3	32.2
+多尺度测试	53.8	32.5	55.7	34.9
集成			59.0	37.4

表 9. 使用 Faster R-CNN 和 ResNet-101 在 MS COCO 上的目标检测改进。

系统	net	data	mAP	areo	bike	bird	boat	瓶子	bus	car	cat	椅子	cow	桌子	dog	马	摩托车	人	植物	羊	sofa	火车	tv
基线	VGG-16	07+12	73.2	76.5	79.0	70.9	65.5	52.1	83.1	84.7	86.4	52.0	81.9	65.7	84.8	84.6	77.5	76.7	38.8	73.6	73.9	83.0	72.6
基线	ResNet-101	07+12	76.4	79.8	80.7	76.2	68.3	55.9	85.1	85.3	89.8	56.7	87.8	69.4	88.3	88.9	80.9	78.4	41.7	78.6	79.8	85.3	72.0
基线+++	ResNet-101	COCO+07+12	85.6	90.0	89.6	87.8	80.8	76.1	89.9	89.9	89.6	75.5	90.0	80.7	89.6	90.3	89.1	88.7	65.4	88.1	85.6	89.0	86.8

表10. PASCAL VOC 2007测试集上的检测结果。基线为Faster R-CNN系统。系统"baseline+++"包含表9中的框精修、上下文信息和多尺度测试。

系统	net	data	mAP	areo	bike	bird	boat	瓶子	bus	car	cat	椅子	cow	表格	dog	马	摩托车	人	植物	羊	sofa	火车	tv
基线	VGG-16	07++12	70.4	84.9	79.8	74.3	53.9	49.8	77.5	75.9	88.5	45.6	77.1	55.3	86.9	81.7	80.9	79.6	40.1	72.6	60.9	81.2	61.5
基线	ResNet-101	07++12	73.8	86.5	81.6	77.2	58.0	51.0	78.6	76.6	93.2	48.6	80.4	59.0	92.1	85.3	84.8	80.7	48.1	77.3	66.5	84.7	65.6
基线+++	ResNet-101	COCO+07++12	83.8	92.1	88.4	84.8	75.9	71.4	86.3	87.8	94.2	66.8	89.4	69.2	93.9	91.9	90.9	89.6	67.9	88.2	76.8	90.3	80.0

表 11: PASCAL VOC 2012 测试集上的检测结果（<http://host.robots.ox.ac.uk:8080/leaderboard/displaylb.php?challengeid=11&compid=4>）。基线为 Faster R-CNN 系统。系统"基线+++"包含表 9 中的边框微调、上下文和多尺度测试。

我们按照 [33] 从金字塔中选择两个相邻的尺度。RoI 池化及后续层在这两个尺度的特征图上执行 [33], , 这两个特征图按 [33] 所示通过 maxout 进行合并。多尺度测试将 mAP 提升了超过 2 个百分点（表 9）。

使用验证数据。接下来我们使用80k+40k trainval集进行训练，使用20k test-dev集进行评估。test-dev集没有公开的基准真值，结果由评估服务器报告。在该设置下，结果为mAP@.5达到55.7%，mAP@[.5,95]达到34.9%（表9）。这是我们的单模型结果。

集成方法。在Faster R-CNN中，系统被设计为同时学习区域候选框和目标分类器，因此可以使用集成方法来提升这两项任务。我们使用一个集成模型来生成区域候选框，候选框的并集由一个区域分类器集成模型进行处理。表9展示了我们基于3个网络集成的结果。在test-dev集上，mAP分别为59.0%和37.4%。该结果在*COCO 2015*检测任务中获得第一名。

PASCAL VOC

我们基于上述模型重新审视了 PASCAL VOC 数据集。使用在 COCO 数据集上的单一模型（表9中 mAP@.5 为 55.7%），我们在 PASCAL VOC 数据集上对该模型进行了微调。同时采用了边框细化、上下文和多尺度测试等改进方法。通过这些手段

	val2	test
GoogLeNet [44] (ILSVRC'14)	-	43.9
我们的单模型（ILSVRC'15）	60.5	58.8
我们的集成模型（ILSVRC'15）	63.6	62.1

表12: 我们在ImageNet检测数据集上的结果（mAP，%）。我们的检测系统是采用表9中改进的Faster R-CNN [32], 使用 ResNet-101。

我们在 PASCAL VOC 2007 上取得了 85.6% 的 mAP（表10），在 PASCAL VOC 2012 上取得了 83.8% 的 mAP（表11）⁶。PASCAL VOC 2012 的结果比之前的最先进结果高出 10 个百分点 [6]。

ImageNet检测

ImageNet 目标检测（DET）任务包含 200 个目标类别，以 mAP@.5 评估精度。我们用于 ImageNet DET 的目标检测算法与表 9 中用于 MS COCO 的算法相同。网络先在 1000 类的 ImageNet 分类数据集上进行预训练，然后在 DET 数据上进行微调。我们按照 [8]将验证集分为两部分（val1/val2）。我们使用 DET 训练集和 val1 集微调检测模型，val2 集用于验证。我们未使用其他 ILSVRC 2015 数据。我们使用 ResNet-101 的单模型达到了

⁶<http://host.robots.ox.ac.uk:8080/anonymous/30J40J.html>, 提交于 2015-11-26。

LOC method	LOC network	testing	LOC error on GT CLS	classification network	top-5 LOC error on predicted CLS
VGG’s [41]	VGG-16	1-crop	33.1 [41]		
RPN	ResNet-101	1-crop	13.3		
RPN	ResNet-101	dense	11.7		
RPN	ResNet-101	dense		ResNet-101	14.4
RPN+RCNN	ResNet-101	dense		ResNet-101	10.6
RPN+RCNN	ensemble	dense		ensemble	8.9

Table 13. Localization error (%) on the ImageNet validation. In the column of “LOC error on GT class” ([41]), the ground truth class is used. In the “testing” column, “1-crop” denotes testing on a center crop of 224×224 pixels, “dense” denotes dense (fully convolutional) and multi-scale testing.

58.8% mAP and our ensemble of 3 models has 62.1% mAP on the DET test set (Table 12). *This result won the 1st place in the ImageNet detection task in ILSVRC 2015*, surpassing the second place by **8.5 points** (absolute).

C. ImageNet Localization

The ImageNet Localization (LOC) task [36] requires to classify and localize the objects. Following [40, 41], we assume that the image-level classifiers are first adopted for predicting the class labels of an image, and the localization algorithm only accounts for predicting bounding boxes based on the predicted classes. We adopt the “per-class regression” (PCR) strategy [40, 41], learning a bounding box regressor for each class. We pre-train the networks for ImageNet classification and then fine-tune them for localization. We train networks on the provided 1000-class ImageNet training set.

Our localization algorithm is based on the RPN framework of [32] with a few modifications. Unlike the way in [32] that is category-agnostic, our RPN for localization is designed in a *per-class* form. This RPN ends with two sibling 1×1 convolutional layers for binary classification (*cls*) and box regression (*reg*), as in [32]. The *cls* and *reg* layers are both in a *per-class* from, in contrast to [32]. Specifically, the *cls* layer has a 1000-d output, and each dimension is *binary logistic regression* for predicting being or not being an object class; the *reg* layer has a 1000×4-d output consisting of box regressors for 1000 classes. As in [32], our bounding box regression is with reference to multiple translation-invariant “anchor” boxes at each position.

As in our ImageNet classification training (Sec. 3.4), we randomly sample 224×224 crops for data augmentation. We use a mini-batch size of 256 images for fine-tuning. To avoid negative samples being dominate, 8 anchors are randomly sampled for each image, where the sampled positive and negative anchors have a ratio of 1:1 [32]. For testing, the network is applied on the image fully-convolutionally.

Table 13 compares the localization results. Following [41], we first perform “oracle” testing using the ground truth class as the classification prediction. VGG’s paper [41] re-

method	top-5 localization err	
	val	test
OverFeat [40] (ILSVRC’13)	30.0	29.9
GoogLeNet [44] (ILSVRC’14)	-	26.7
VGG [41] (ILSVRC’14)	26.9	25.3
ours (ILSVRC’15)	8.9	9.0

Table 14. Comparisons of localization error (%) on the ImageNet dataset with state-of-the-art methods.

ports a center-crop error of 33.1% (Table 13) using ground truth classes. Under the same setting, our RPN method using ResNet-101 net significantly reduces the center-crop error to 13.3%. This comparison demonstrates the excellent performance of our framework. With dense (fully convolutional) and multi-scale testing, our ResNet-101 has an error of 11.7% using ground truth classes. Using ResNet-101 for predicting classes (4.6% top-5 classification error, Table 4), the top-5 localization error is 14.4%.

The above results are only based on the *proposal network* (RPN) in Faster R-CNN [32]. One may use the *detection network* (Fast R-CNN [7]) in Faster R-CNN to improve the results. But we notice that on this dataset, one image usually contains a single dominate object, and the proposal regions highly overlap with each other and thus have very similar RoI-pooled features. As a result, the image-centric training of Fast R-CNN [7] generates samples of small variations, which may not be desired for stochastic training. Motivated by this, in our current experiment we use the original R-CNN [8] that is RoI-centric, in place of Fast R-CNN.

Our R-CNN implementation is as follows. We apply the per-class RPN trained as above on the training images to predict bounding boxes for the ground truth class. These predicted boxes play a role of class-dependent proposals. For each training image, the highest scored 200 proposals are extracted as training samples to train an R-CNN classifier. The image region is cropped from a proposal, warped to 224×224 pixels, and fed into the classification network as in R-CNN [8]. The outputs of this network consist of two sibling fc layers for *cls* and *reg*, also in a per-class form. This R-CNN network is fine-tuned on the training set using a mini-batch size of 256 in the RoI-centric fashion. For testing, the RPN generates the highest scored 200 proposals for each predicted class, and the R-CNN network is used to update these proposals’ scores and box positions.

This method reduces the top-5 localization error to 10.6% (Table 13). This is our single-model result on the validation set. Using an ensemble of networks for both classification and localization, we achieve a top-5 localization error of 9.0% on the test set. This number significantly outperforms the ILSVRC 14 results (Table 14), showing a 64% relative reduction of error. *This result won the 1st place in the ImageNet localization task in ILSVRC 2015*.

LOC 方法	LOC 网络	测试	LOC 误差在 GT CLS 上	分类网络	top-5 LOC 误差基于预测的 CLS
VGG 的 [41]	VGG-16	单裁剪	33.1 [41]		
RPN	ResNet-101	1-crop	13.3		
RPN	ResNet-101	dense	11.7		
RPN	ResNet-101	dense		ResNet-101	14.4
RPN+RCNN	ResNet-101	dense		ResNet-101	10.6
RPN+RCNN	集成	密集		集成	8.9

表13: ImageNet验证集上的定位误差(%). 在“GT类别上的定位误差” ([41])一列中, 使用真实类别进行评估; 在“testing”一列中, “1-crop”表示对 224×224 像素的中心裁剪区域进行测试, “dense”表示密集 (全卷积) 和多尺度测试。

58.8% 的 mAP, 3 个模型的集成在 DET 测试集上达到了 62.1% 的 mAP (表 12)。该结果在 *ILSVRC 2015* 的 *ImageNet* 检测任务中获得了第一名, 以**8.5 个百分点** (绝对值) 的优势超过第二名。

C. ImageNet 图像定位

ImageNet定位 (LOC) 任务 [36] 要求对物体进行分类和定位。沿用 [40, 41], 我们假设先使用图像级分类器预测图像的类别标签, 而定位算法仅负责根据预测的类别预测边界框。我们采用“逐类回归” (PCR) 策略 [40, 41], , 为每个类别学习一个边界框回归器。我们先在ImageNet分类任务上预训练网络, 然后针对定位任务进行微调。我们在提供的1000类ImageNet训练集上训练网络。

我们的定位算法基于 [32] 的RPN框架, 并进行了少量修改。与[32] 中类别无关的方式不同, 我们用于定位的RPN采用逐类别的形式设计。该RPN以两个并列的 1×1 卷积层结尾, 分别用于二分类 (*cls*) 和边界框回归 (*reg*), 如 [32]所示。 *cls* 层和*reg* 层均采用逐类别形式, 这与 [32] 不同。具体而言, *cls*层具有1000维输出, 每一维都是二元逻辑回归, 用于预测是否属于某一目标类别; *reg* 层具有 1000×4维输出, 由针对1000个类别的边界框回归器组成。如 [32],所示, 我们的边界框回归以每个位置上的多个平移不变的"锚点"框为参考。

与我们的ImageNet分类训练 (第3.4节) 一样, 我们随机采样 224×224 裁剪区域进行数据增强。微调时, 我们使用每批256张图像的小批量。为避免负样本占主导, 对每张图像随机采样8个锚框, 其中采样的正负锚框比例为1:1 [32]。测试时, 网络以全卷积方式应用于整幅图像。

表13比较了定位结果。按照[41], 的思路, 我们首先使用真实类别作为分类预测来进行"oracle"测试。VGG的论文 [41] 中

方法	top-5定位错误率	
	val	test
OverFeat [40] (ILSVRC’13)	30.0	29.9
GoogLeNet [44] (ILSVRC’14)	-	26.7
VGG [41](ILSVRC’14)	26.9	25.3
本文方法 (ILSVRC’15)	8.9	9.0

表14. 在ImageNet数据集上与最先进方法的定位错误率 (%) 比较。

center-crop误差为33.1% (表13), 使用真实类别。在相同设置下, 我们使用ResNet-101网络的RPN方法将center-crop误差显著降低至13.3%。这一比较展示了我们框架的出色性能。通过密集 (全卷积) 和多尺度测试, 我们的ResNet-101使用真实类别时误差为 11.7%。使用ResNet-101预测类别 (top-5分类误差为 4.6%, 表4), top-5定位误差为14.4%。

以上结果仅基于Faster R-CNN中的候选区域生成网络 (RPN) [32]。可以使用Faster R-CNN中的检测网络 (Fast R-CNN [7]) 来改进结果。但我们注意到, 在该数据集中, 一张图像通常只包含一个主要目标, 且候选区域之间高度重叠, 因此其RoI池化特征非常相似。结果, Fast R-CNN [7]基于图像中心的训练产生的样本变化很小, 这可能不利于随机训练。受此启发, 在当前实验中我们使用基于RoI中心的原始R-CNN [8]来替代Fast R-CNN。

我们的R-CNN实现如下。我们将上述训练好的逐类RPN应用于训练图像, 以预测真实类别的边界框。这些预测框起到类别相关候选区域的作用。对于每张训练图像, 提取得分最高的200个候选区域作为训练样本来训练 R-CNN分类器。图像区域从候选区域中裁剪出来, 缩放到 224×224像素, 然后按照R-CNN [8]的方式输入分类网络。该网络的输出由两个并列的全连接层组成, 分别用于分类和回归, 同样采用逐类形式。该R-CNN网络在训练集上使用批大小为256的mini-batch, 以RoI中心的方式进行微调。在测试时, RPN为每个预测类别生成得分最高的200个候选区域, 然后使用R-CNN网络更新这些候选区域的得分和框位置。

该方法将 top-5 定位误差降低至 10.6% (表 13)。这是我们在验证集上的单模型结果。同时使用分类和定位的网络集成, 我们在测试集上取得了 9.0% 的 top-5 定位误差。这一数值显著优于 ILSVRC 2014 的结果 (表 14), 误差相对降低了 64%。该结果在 *ILSVRC 2015* 的 *ImageNet* 定位任务中获得了第 1 名。